

Aprendizaje Automático 1

Esp. Ing. Joel Spak

Agosto 2025

Contents

1	Introducción al aprendizaje automático	6
1.1	Aprendizaje como rama de IA	6
1.2	Aprendizaje supervisado, no supervisado y por refuerzo	6
1.2.1	Aprendizaje supervisado -supervised learning-	6
1.2.2	Aprendizaje no supervisado -unsupervised learning-	7
1.2.3	Aprendizaje por refuerzo -reinforcement learning-	7
1.3	Tipos de tareas en aprendizaje supervisado	8
1.3.1	Regresión	8
1.3.2	Clasificación	8
1.4	Ciclo de vida	9
1.4.1	Recolección de datos	9
1.4.2	Análisis exploratorio de datos	10
1.4.3	Selección y entrenamiento de modelos	10
1.4.4	Evaluación de modelos	10
1.4.5	Despliegue -puesta en producción-	10
1.4.6	Monitoreo	11
1.5	Componentes de un algoritmo de aprendizaje	11
1.5.1	Función de pérdida -loss function-	11
1.5.2	Optimizador	11
1.5.3	Métricas	12
1.5.4	Parámetros e hiperparámetros.	12
1.6	Evaluación de modelos de aprendizaje	13
1.6.1	Sesgo y varianza	13
1.6.2	Ajuste de modelos	13
1.6.3	Validación cruzada	14
1.7	Fuga de datos -data leakage-	17
1.7.1	Contaminación train - test	18
1.7.2	Prevención	18
2	Modelos lineales para regresión	19
2.1	Regresión lineal simple	19
2.2	Regresión lineal múltiple	20
2.3	Métricas para regresión	21
2.3.1	Error cuadrático medio (Mean Squared Error o MSE)	21
2.3.2	Raíz del error cuadrático medio (Root Mean Squared Error o RMSE)	21
2.3.3	Error absoluto medio (Mean Absolute Error o MAE)	21
2.3.4	Error absoluto medio porcentual (Mean Absolute Percentage Error o MAPE)	22
2.3.5	R^2 Score	22
2.3.6	R^2 ajustado	22
2.4	Gradiente descendiente	22
2.4.1	Motivación	22
2.4.2	Técnica del gradiente descendiente	23

2.4.3	Tasa de aprendizaje -learning rate-	24
2.4.4	Gradiente descendiente estocástico (SGD)	25
2.4.5	Gradiente descendiente Mini-batch	25
2.4.6	Comparación GD, SGD, mini-batch GD	25
2.5	Regularización	26
2.5.1	Motivación: reducción de la varianza en un modelo de regresión lineal	26
2.5.2	Regularización L2: Ridge	26
2.5.3	Regularización L1: Lasso	27
2.5.4	L1 vs L2	27
2.5.5	Generalización	28
2.5.6	ElasticNet	29
3	Modelos lineales para clasificación	30
3.1	Clasificación binaria vs multiclase	30
3.2	La regresión lineal como clasificador	31
3.3	Regresión logística	32
3.4	Chances (odds)	32
3.5	Logaritmo de las chances (log-odds o logit)	33
3.6	¿Por qué se llama regresión logística?	34
3.7	Estimación de los coeficientes de la regresión logística	34
3.7.1	Función de costo por máxima verosimilitud	34
3.7.2	Entropía binaria cruzada	35
3.7.3	Regularización	35
3.7.4	Estimación por gradiente descendiente	36
3.8	Problemas multiclase	37
4	Métricas para clasificación	39
4.1	Matriz de confusión	39
4.2	Accuracy	40
4.3	Recall (sensibilidad o tasa de verdaderos positivos)	40
4.4	Precision (precisión)	41
4.5	F1-score	41
4.6	Promedios: micro, macro y ponderado	42
4.6.1	Micro promedio	42
4.6.2	Macro promedio	42
4.6.3	Promedio ponderado	42
4.7	Métricas personalizadas	43
4.8	Curva ROC y AUC	43
4.8.1	AUC - Área bajo la curva	44
5	Balanceo de clases	45
5.1	Técnicas para balancear clases	45
5.2	Métricas adecuadas	46
6	Explicabilidad de modelos	47

7	Comparación de modelos y ajuste fino	48
7.1	Baseline models	48
7.1.1	Tipos de modelos baseline	48
7.1.2	Comparación con modelos complejos	48
7.2	Criterios para comparación de modelos	48
8	Ajuste fino	50
8.1	Optimización de hiperparámetros	50
8.2	Métodos de búsqueda	50
9	Introducción a redes neuronales	53
9.1	Del aprendizaje automático al aprendizaje profundo	53
9.2	Los modelos lineales no son suficientes	53
9.3	La neurona artificial	54
9.4	Conexión entre neuronas y capas	54
9.5	Entrenamiento de una red neuronal	54
9.6	Funciones de pérdida	55
9.7	Optimizadores	55
9.8	Arquitecturas de modelos	56
9.9	Ecuaciones feed-forward	57
9.10	Ecuaciones back-propagation	57
9.11	Funciones de activación	59
9.11.1	Función escalón	60
9.11.2	Función sigmoide	60
9.11.3	Tangente hiperbólica (tanh)	61
9.11.4	ReLU (Rectified Linear Unit)	61
9.11.5	Leaky ReLU	62
9.12	Regularización	63
9.12.1	Penalizaciones de norma en los parámetros	63
9.12.2	Detención temprana (Early Stopping)	63
9.12.3	Aumento de datos (Data Augmentation)	64
9.12.4	Regularización por ruido	64
9.12.5	Ensamblado de modelos (Ensemble Learning)	64
9.12.6	Dropout	64
9.12.7	Ejemplos adversariales	66
9.13	Ajuste de hiperparámetros	66
10	MLOps	67
10.1	Entornos de Desarrollo y Producción	67
10.1.1	Versionado de modelos	68
10.1.2	Despliegue y automatización	68
10.2	Monitoreo y mantenimiento	69
10.2.1	Deriva de datos (data drift)	69
10.2.2	Alertas y retrain	70
10.3	Niveles de MLOps	70

10.3.1	Integración continua	70
10.3.2	Entrega continua	70
10.3.3	Nivel 0	71
10.3.4	Nivel 1	71
10.3.5	Nivel 2	74

1 Introducción al aprendizaje automático

1.1 Aprendizaje como rama de IA

La Inteligencia Artificial es un campo de la informática que busca desarrollar sistemas capaces de realizar tareas que normalmente requieren inteligencia humana. Dentro de la IA existen varias ramas. Entre ellas se pueden destacar:

- **Procesamiento del Lenguaje Natural (NLP)**: permite a las máquinas comprender, interpretar y generar lenguaje humano. Es el caso de los chatbots, asistentes virtuales, traducción entre distintos idiomas, estudio de sentiment analysis, entre otros.
- **Visión por Computadora (Computer Vision)**: facilita el análisis y la interpretación de imágenes y videos. Algunos ejemplos son el reconocimiento facial, los vehículos autónomos (que requieren cámaras para detectar peatones, señales de tránsito, etc), control de calidad mediante imágenes, reconocimiento de objetos, entre otros.
- **Aprendizaje Automático (Machine Learning, ML)**: se basa en la creación de algoritmos que pueden aprender patrones a partir de datos y mejorar su rendimiento sin ser programados explícitamente, es decir, no siguen reglas predefinidas. Estos algoritmos pueden hacer predicciones o tomar decisiones basadas en datos.

Aprender es, acorde a la RAE, "adquirir el conocimiento de algo por medio del estudio o de la experiencia" [6]. De aprender pueden surgir tres importantes capacidades: recordar, adaptarse y generalizar. **Recordar** aquellas veces en las que se ha estado en la misma situación que la actual (observaciones), en las que hemos intentado una acción (variables) y si el resultado fue bueno (output) se puede intentar una acción similar, en caso contrario, intentar otra acción distinta, **adaptándose**. **Generalizar** hace referencia a reconocer la similitud entre situaciones para interpretar que herramientas aprendidas son las más útiles en cada caso [11] [4].

En aprendizaje de máquina, aprender implica mejorar el desempeño en una tarea a medida que se obtiene más experiencia o datos. La máquina puede aprender a través de modificar o adaptar sus acciones (que pueden ser realizar predicciones, controlar un robot, etc). Un sistema aprende si su rendimiento mejora con el tiempo al procesar nueva información.

1.2 Aprendizaje supervisado, no supervisado y por refuerzo

Se puede clasificar a un problema de aprendizaje automático de tres formas distintas: supervisado, no supervisado y por refuerzo.

1.2.1 Aprendizaje supervisado -supervised learning-

Se entrena al modelo con datos etiquetados (es decir, cada entrada tiene una salida conocida) para resolver una tarea en específico. Ejemplo: clasificación de correos electrónicos en "spam" o "no spam"; estimación de ventas para el próximo año.

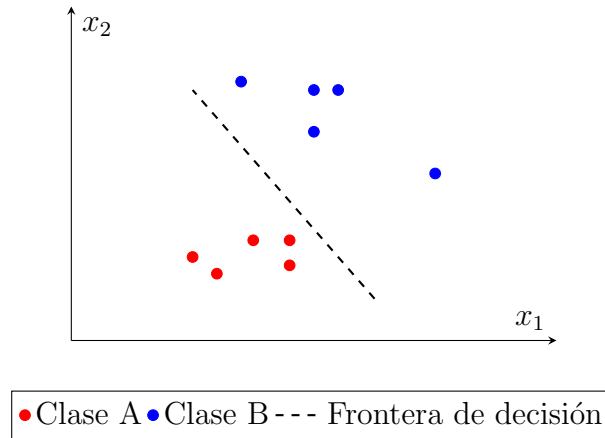


Figura 1: Ejemplo de clasificación binaria en aprendizaje supervisado (datos etiquetados), con frontera de decisión determinada por un algoritmo de aprendizaje.

1.2.2 Aprendizaje no supervisado -unsupervised learning-

Se trabaja con datos sin etiquetas y el modelo debe encontrar patrones ocultos por sí solo. Ejemplos: segmentación de clientes en un negocio, detección de anomalías. Dentro de problemas de aprendizaje supervisado, son útiles como parte del preprocesamiento de datos para imputar datos faltantes, reducción de dimensionalidad y para generar nuevas features basadas en los mencionados patrones.

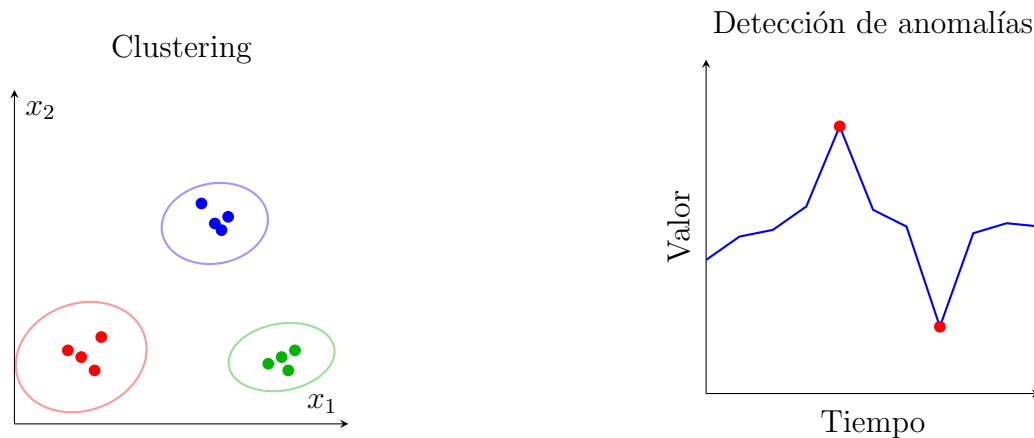


Figura 2: Ejemplos de aprendizaje no supervisado: clustering (izquierda) y detección de anomalías en una serie temporal (derecha).

1.2.3 Aprendizaje por refuerzo -reinforcement learning-

El modelo aprende a tomar decisiones mediante un sistema de recompensas y penalizaciones. El ejemplo más común es el de algún juego, donde el agente de IA comienza su entrenamiento al jugar de forma prácticamente aleatoria y en la medida que se le va asignando recompensas y castigos puede ir mejorando su estrategia.

1.3 Tipos de tareas en aprendizaje supervisado

1.3.1 Regresión

Un problema de regresión ocurre cuando la variable a predecir es numérica y continua. Es decir, queremos estimar un valor dentro de un rango infinito de posibilidades.

- Predecir el precio de una casa en función de su tamaño, ubicación y número de habitaciones.
- Estimar la temperatura de una ciudad en función de datos meteorológicos anteriores.
- Pronosticar las ventas futuras de una tienda basándose en el historial de ventas.
- Predecir el valor de una acción de Apple para el día de mañana, en función de retornos de días anteriores e indicadores estadísticos.

1.3.2 Clasificación

Un problema de clasificación ocurre cuando la variable a predecir es categórica, es decir, pertenece a un conjunto finito de clases o etiquetas. El objetivo es asignar correctamente una observación a una de dichas clases.

- Determinar si un correo electrónico es *spam* o *no spam* en base a su contenido.
- Diagnosticar si un paciente tiene una enfermedad en particular (por ejemplo, diabetes) según sus síntomas y resultados clínicos.
- Clasificar una imagen como gato, perro o ave utilizando técnicas de visión por computadora.
- Predecir si un cliente abandonará un servicio (churn) según su historial de uso y comportamiento.

Mientras que en clasificación buscamos responder preguntas como “¿A qué categoría pertenece esta observación?”, en regresión respondemos preguntas del tipo “¿Qué valor tendrá esta variable?”. Ambos tipos de problemas requieren enfoques, métricas y algoritmos distintos, aunque algunos modelos como los árboles de decisión o redes neuronales pueden adaptarse a ambos escenarios. Los modelos lineales también pueden adaptarse, aunque requerirá efectuar algunas transformaciones para llegar de una regresión lineal (problema de regresión) a una regresión logística (problema de clasificación).

Si se piensa en un ejemplo como el de predecir si el día de mañana puede llover, se piensa automáticamente en un problema de clasificación donde la respuesta final será “Si” o “No”. Pero es común ver asociada una probabilidad de pertenencia de la observación a cada una de las dos clases. Un problema de clasificación puede pensarse como uno de regresión cuando se piensa en una función de probabilidad, lo cual es sumamente interesante a la hora de predecir para acompañar las predicciones, pero cuya diferencia principal con el problema de regresión es que no se cuenta con las etiquetas reales de probabilidad, sólo se tienen etiquetas reales de clases.

1.4 Ciclo de vida

El ciclo de vida de un proyecto de ML refiere a todas las etapas necesarias para transformar datos en valor mediante modelos predictivos y productivos. A continuación, se describen sus principales procesos.

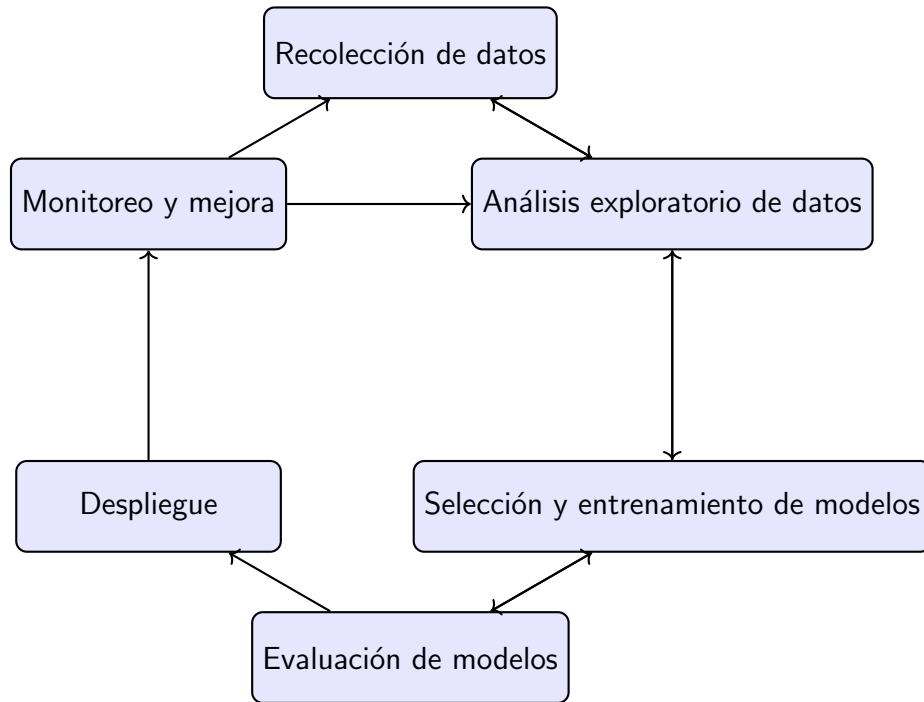


Figura 3: Ciclo de vida de un proyecto de Machine Learning

1.4.1 Recolección de datos

A lo largo de este curso trabajaremos con conjuntos de datos ya preparados y disponibles para descargar, lo que facilita la tarea de recolección. Sin embargo, en aplicaciones reales no suele ser tan simple. Cuando se aborda un problema nuevo, la recolección de datos debe ir de la mano con la definición de las variables relevantes. Es recomendable comenzar con un conjunto de datos pequeño, pero que contenga todas las variables potencialmente útiles. Este primer conjunto puede servir para realizar pruebas preliminares, seleccionar características y luego orientar la recolección completa y definitiva.

Usualmente, los datos que se necesitan son abundantes pero difíciles de acceder o combinar. Pueden estar almacenados en distintos formatos, en bases de datos separadas o requerir mediciones específicas que llevan tiempo y recursos. A esto se suma la necesidad de verificar su calidad: los datos deben estar limpios, sin errores graves ni valores faltantes que afecten el análisis posterior.

En el caso del aprendizaje supervisado, también es necesario contar con los valores objetivo o etiquetas, lo cual puede implicar el trabajo de especialistas y un esfuerzo adicional en la anotación del conjunto de datos.

Por último, debe tenerse en cuenta la cantidad de datos. Si bien los modelos de aprendizaje automático requieren grandes volúmenes de información para aprender patrones útiles, trabajar con conjuntos muy extensos también incrementa los costos computacionales. Por eso, uno de los primeros desafíos es encontrar el equilibrio justo entre calidad, cantidad y viabilidad de los datos recolectados.

1.4.2 Análisis exploratorio de datos

En esta fase se analiza de forma preliminar el conjunto de datos para entender su estructura, identificar patrones, valores atípicos y datos faltantes y codificar variables en caso de ser necesario. El objetivo es trabajar y transformar los datos para que puedan ser input de un algoritmo de aprendizaje, y tener una idea del poder predictivo de cada variable y qué tipo de algoritmo puede ajustarse correctamente (por ejemplo, un algoritmo lineal no será capaz de capturar relaciones cuadráticas entre variables, por lo que se requeriría transformar las variables o usar algoritmos que sí permitan capturar esas relaciones).

1.4.3 Selección y entrenamiento de modelos

La selección de modelos va de la mano del análisis exploratorio, donde debería poder observarse, de forma aproximada, qué algoritmos pueden ser útiles. También se deberá seleccionar los parámetros del modelo que deban setearse previo a utilizar el algoritmo, que a lo largo del curso se llamarán "hiperparámetros".

Es necesario contar con un conjunto de datos de **entrenamiento** para ajustar el algoritmo de aprendizaje. En el caso de que se emplee una variación de los hiperparámetros, se debe utilizar un conjunto de **validación** para evaluar el desempeño del modelo con cada configuración y seleccionar los valores óptimos. Finalmente, un conjunto de **prueba** independiente permite estimar el rendimiento real del modelo sobre datos no vistos.

1.4.4 Evaluación de modelos

Una vez entrenado el modelo, es necesario evaluar su desempeño para determinar si cumple con los objetivos del problema planteado. La evaluación debe realizarse utilizando el conjunto de **prueba**, que contiene datos que el modelo no ha visto previamente. Esto permite estimar su capacidad de generalización a nuevos casos.

Los criterios de evaluación dependerán del tipo de problema, pero siempre a través de una métrica, que se discute en la sección 1.5.3.

Además de los valores numéricos, puede ser útil visualizar los resultados mediante gráficos, como la matriz de confusión en clasificación o los gráficos de residuos en regresión. Estas herramientas permiten identificar patrones de error sistemáticos que podrían corregirse con un ajuste del modelo o del conjunto de datos.

1.4.5 Despliegue -puesta en producción-

Cuando un modelo alcanza un nivel de desempeño satisfactorio, el siguiente paso es ponerlo en funcionamiento en un entorno real, lo que se conoce como **despliegue** o **puesta**

en producción. Esto puede implicar integrarlo en una aplicación, en un sistema automatizado de toma de decisiones, o bien proporcionar sus predicciones como un servicio accesible vía API.

Durante esta etapa, es importante considerar aspectos como la eficiencia computacional del modelo, la latencia en las predicciones, la facilidad de mantenimiento y actualización, y la seguridad y privacidad de los datos que se procesan.

Además, deben establecerse procedimientos claros para la actualización del modelo a medida que se disponga de nuevos datos, asegurando que el sistema siga siendo relevante y efectivo con el tiempo.

1.4.6 Monitoreo

Una vez en producción, el modelo debe ser **monitoreado** de manera continua para asegurar que su rendimiento se mantenga estable. Con el tiempo, los datos pueden cambiar (fenómeno conocido como *drift*, sección 10.2.1) y el modelo podría volverse menos preciso o incluso tomar decisiones erróneas.

El monitoreo implica recolectar métricas de uso y desempeño, comparar predicciones con resultados reales (cuando estén disponibles) y generar alertas ante comportamientos anómalos.

En caso de detectar una degradación en el rendimiento, puede ser necesario volver a entrenar el modelo con nuevos datos, ajustar los hiperparámetros, o incluso rediseñar el enfoque del problema, partiendo desde cero.

1.5 Componentes de un algoritmo de aprendizaje

1.5.1 Función de pérdida -loss function-

Cuantifica el error entre las predicciones del modelo y los valores reales. Su objetivo es guiar el proceso de aprendizaje: cuanto mayor sea el error, mayor será la pérdida, y viceversa. De forma tal que el aprendizaje debe darse para disminuir el valor de esta función (también llamada función de costo).

El tipo de función de pérdida depende del tipo de problema. En regresión, se suelen utilizar el *error cuadrático medio* (MSE) o el *error absoluto medio* (MAE). En clasificación, donde el problema pasa por modelar una probabilidad de pertenencia a una clase, se emplean funciones como la *entropía cruzada* (cross-entropy) o la *hinge loss* para modelos como las máquinas de soporte vectorial.

Durante el entrenamiento, el algoritmo intenta minimizar esta función de pérdida para encontrar los mejores parámetros del modelo.

1.5.2 Optimizador

Es el mecanismo que utiliza el algoritmo para ajustar los parámetros internos del modelo con el fin de minimizar la función de pérdida. Los optimizadores aplican métodos matemáticos para encontrar, de manera eficiente, el mínimo de la función de pérdida en el espacio de parámetros.

Uno de los métodos más comunes es el *descenso por gradiente* (gradient descent), que actualiza los parámetros en la dirección opuesta al gradiente de la función de pérdida. Existen variantes más sofisticadas como *Stochastic Gradient Descent* (SGD), *Adam*, *RM-SProp* o *Adagrad*, que mejoran la velocidad de convergencia o adaptan la tasa de aprendizaje durante el entrenamiento (que es un hiperparámetro de la estrategia). Más sobre optimizadores en la sección 9.7.

La elección del optimizador puede influir notablemente en la calidad del modelo y en el tiempo necesario para entrenarlo.

1.5.3 Métricas

Las **métricas** son herramientas para evaluar el desempeño del modelo y pueden diferir de la función de pérdida. Mientras que la pérdida es utilizada para entrenar el modelo, las métricas se utilizan para interpretar y comparar modelos.

En problemas de clasificación, las métricas comunes incluyen:

- accuracy (precisión global)
- precision
- recall
- F1-score
- AUC-ROC (para evaluar la capacidad de discriminación del modelo).
- más en sección 4.

En problemas de regresión, las métricas típicas son:

- MAE
- MSE
- RMSE
- R cuadrado (R^2), que indica qué proporción de la varianza es explicada por el modelo.

1.5.4 Parámetros e hiperparámetros.

Los **parámetros** son los valores que el modelo aprende automáticamente a partir de los datos durante el proceso de entrenamiento. Por ejemplo, en una regresión lineal, los coeficientes de las variables son parámetros que se ajustan para minimizar la pérdida.

Por otro lado, los **hiperparámetros** son configuraciones predefinidas antes del entrenamiento que afectan cómo se entrena el modelo. Algunos ejemplos son:

- La tasa de aprendizaje del optimizador.
- La cantidad de capas o neuronas en una red neuronal.

- La profundidad máxima de un árbol de decisión.
- El número de vecinos en un algoritmo K-nearest-neighbours.

A diferencia de los parámetros, los hiperparámetros no se aprenden automáticamente, sino que deben seleccionarse mediante validación o búsqueda sistemática, como *grid search* o *random search* (sección 8.2). Una buena elección de hiperparámetros puede marcar una gran diferencia en el rendimiento final del modelo.

1.6 Evaluación de modelos de aprendizaje

1.6.1 Sesgo y varianza

El **sesgo** y la **varianza** son dos conceptos que ayudan a entender el tipo de error que está cometiendo el modelo y cómo podría mejorarse.

El **sesgo** se refiere a los errores sistemáticos que comete un modelo al hacer suposiciones incorrectas sobre la relación entre las variables. Un modelo con alto sesgo es generalmente demasiado simple y no logra capturar la complejidad del fenómeno. Esto se conoce como *subajuste* (*underfitting*).

La **varianza**, en cambio, representa la sensibilidad del modelo a pequeñas variaciones en los datos de entrenamiento. Un modelo con alta varianza se ajusta demasiado a los datos de entrenamiento, incluso al ruido, y por lo tanto pierde capacidad de generalización. Esto se conoce como *sobreajuste* (*overfitting*).

El objetivo es encontrar un equilibrio adecuado entre sesgo y varianza (*bias-variance trade-off*) y por lo tanto lograr un modelo que generalice bien a datos nuevos con la mejor métrica posible. Situación que se conoce como *ajuste balanceado* (*balanced fit*).

1.6.2 Ajuste de modelos

El ajuste de un modelo de aprendizaje automático se refiere a qué tan bien logra representar la relación entre las variables de entrada y salida en los datos. Dependiendo del equilibrio entre sesgo y varianza, pueden darse tres situaciones principales:

- **subajuste (underfitting):** Ocurre cuando el modelo es demasiado simple para capturar los patrones de los datos. Esto suele estar asociado a un sesgo alto y una varianza baja. El modelo tiene un rendimiento deficiente tanto en los datos de entrenamiento como en los de prueba.
- **ajuste adecuado (well-balanced):** El modelo logra un buen equilibrio entre sesgo y varianza. Captura los patrones relevantes de los datos sin sobreajustarse al ruido. Tiene un buen rendimiento en entrenamiento y generaliza bien a nuevos datos.
- **sobreajuste (overfitting):** Sucede cuando el modelo es demasiado complejo y se ajusta demasiado a los datos de entrenamiento, incluyendo el ruido. Esto se traduce en una varianza alta y un sesgo bajo. El modelo presenta un alto rendimiento en el entrenamiento, pero falla al generalizar a datos nuevos.

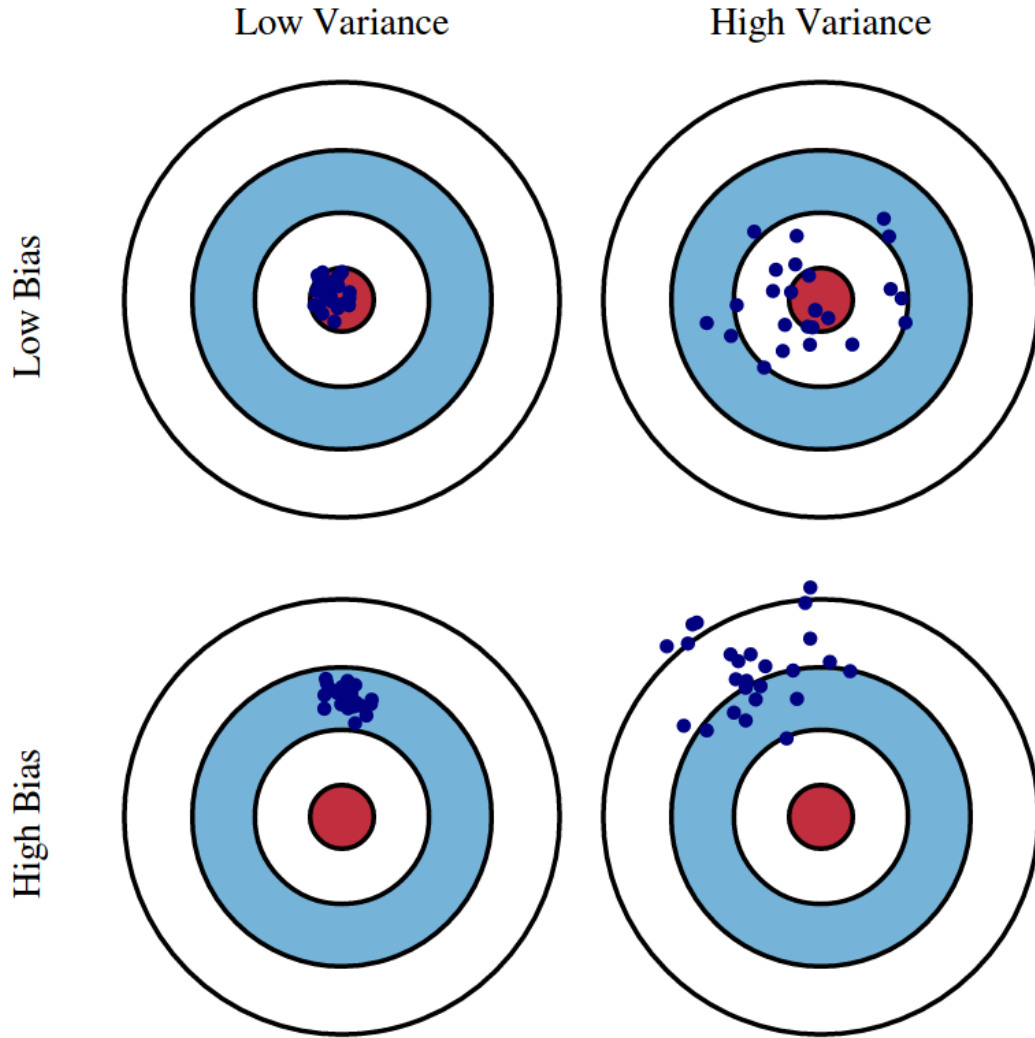


Figura 4: Gráfico de precisión -donde el target representa un error bajo- donde se ejemplifican los sesgos y varianzas bajos y elevados.

1.6.3 Validación cruzada

Es una técnica utilizada para evaluar la capacidad de generalización de un modelo de aprendizaje automático. Permite estimar su desempeño sobre datos no vistos, con el objetivo de evitar tanto el sobreajuste como el subajuste.

El método más simple consiste en el *train-test split*, y uno de los más utilizados es la **validación cruzada k-folds**.

Train-test split Una forma simple de evaluar el rendimiento de un modelo es dividir el conjunto de datos en dos subconjuntos: entrenamiento (*train*) y prueba (*test*). El modelo se entrena con los datos de entrenamiento y se evalúa con los datos de prueba.

Esta estrategia permite medir la capacidad de generalización del modelo. Una división común es usar el 70% u 80% de los datos para entrenamiento y el resto para prueba. Usualmente esta división es de forma aleatoria, excepto en casos donde deba respetarse

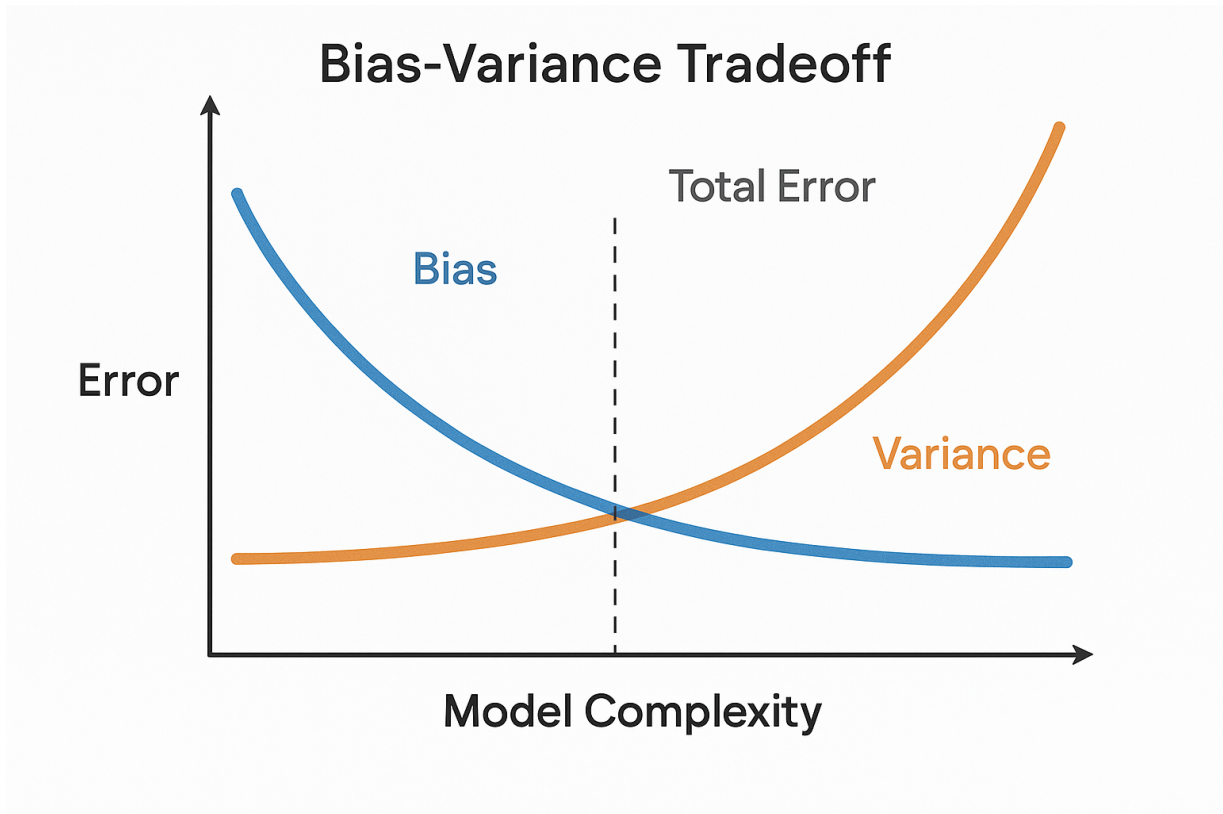


Figura 5: Curva de error por sesgo y varianza vs complejidad del malgoritmo de aprendizaje.

la temporalidad de los datos, donde suele usarse los primeros datos para entrenar y los posteriores para testing.

Puede además usarse una porción de los datos como conjunto de validación, para la optimización de hiperparámetros, como se discutió en la sección 1.4.3.

Sin embargo, este enfoque tiene una limitación importante: el resultado depende de cómo se realizó la división. Si los datos no son representativos o si hay mucho azar en la partición, la evaluación puede ser poco fiable. Por eso, en muchos casos se prefiere usar validación cruzada k-folds.

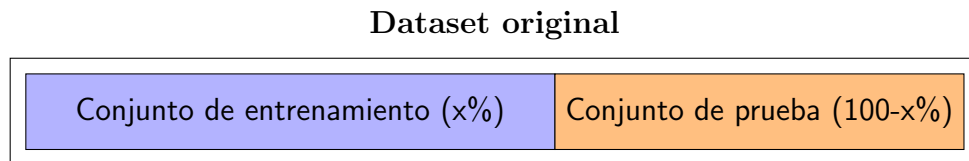


Figura 6: Train-test split

La **Validación cruzada k-folds** consiste en:

- dividir el conjunto de datos en k subconjuntos (o *folds*) del mismo tamaño.

- entrenar el modelo k veces, cada vez utilizando $k - 1$ subconjuntos para entrenar y el subconjunto restante para validar.
- promediar las métricas de rendimiento obtenidas en cada iteración para obtener una estimación global.

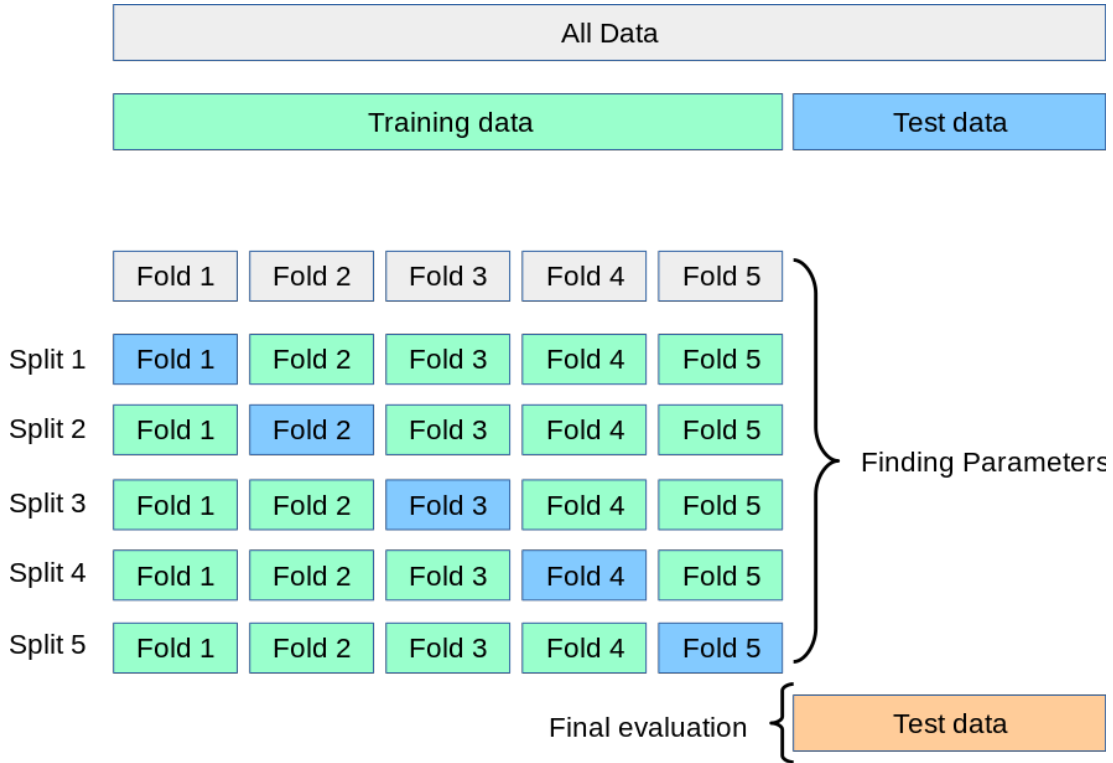


Figura 7: Validación cruzada k -folds para $k = 5$. Se realizan k instancias de entrenamiento variando el conjunto de validación [14].

Este enfoque permite aprovechar todos los datos tanto para entrenamiento como para validación. Se mitiga el riesgo de depender de una sola división de los datos y se obtiene un resultado mucho más robusto. La varianza (o desvío estándar) de los k resultados se puede utilizar como medida del ajuste discutido en 1.6.2. Además, el hecho de poder usar el fold sobrante para validación, lo convierte en un método extremadamente utilizado para el ajuste de hiperparámetros.

Si bien k suele tomar valores entre 5 y 10 (para entrenar con entre un 80 y un 90% de los datos), una variante extrema es la **validación leave-one-out (LOO)**, que es un caso donde k es igual al número de observaciones. Aunque precisa, puede ser computacionalmente costosa, dado que requiere entrenar tantos modelos como datos se tengan. Los resultados de validación o prueba suelen ser ruidosos, ya que son sobre un sólo dato, y en todos los casos se entrena un modelo con un $(k - 1)/k$ datos donde, si k es suficientemente grande, todos estos modelos son virtualmente el mismo. Puede ser un estimador de la varianza sobre los datos de prueba [10].

1.7 Fuga de datos -data leakage-

Cuando el conjunto de datos contiene datos relevantes, pero no se pueden obtener datos similares cuando los modelos se utilizan para predicciones (estado de producción), se produce una fuga (o filtración) de datos. Esto da como resultado un gran éxito en el conjunto de datos de entrenamiento (y posiblemente incluso en la precisión de la validación/testeo), pero una falta de rendimiento en producción, o incluso, la imposibilidad de llevar a cabo el experimento.

Se produce cuando durante el aprendizaje se introduce información sobre la etiqueta o el número de destino que no sería legalmente accesible durante el uso real. **Ejemplo 1: si**

la verdadera etiqueta de un conjunto de datos se incluyera como característica en el modelo. En este caso la predicción del modelo debería ser perfecta, ya que tendría una variable explicativa que se correlaciona perfectamente con la salida. Sin embargo, en la puesta en producción, no es una variable disponible. **Ejemplo 2: predicción de**

consumo de energía eléctrica, con temperatura como factor determinante. El entrenamiento se realiza con datos reales de temperatura, pero a la hora de pronosticar solo se cuenta con previsiones de temperatura. Los posibles errores de previsión de cualquier variable meteorológica puede influir fuertemente en las métricas del modelo en producción. **Ejemplo 3: sitio web de venta al por menor y necesitas crear un clasificador**

para predecir si un usuario se quedará a leer otra página o se irá. Si el clasificador predice que van a irse, el sitio web puede mostrar algo que los anime a quedarse y comprar. La duración general de la sesión del usuario o el número total de páginas vistas durante su visita al sitio son ejemplos de características que contienen información filtrada. Durante la fase de postprocesamiento de los datos del registro de visita, por ejemplo, este total se agrega con frecuencia como una nueva columna. Esta característica contiene información sobre las visitas futuras del usuario, como cuántos viajes más hará el usuario. En una implementación del mundo real, es imposible determinarlo. La característica de duración total de la sesión podría ser reemplazada por una característica de visita a página en sesión, que solo sabe cuántas páginas se han visitado hasta el momento en la sesión, no cuántas quedan. **Ejemplo 4: predicción de diagnóstico médico con imágenes.** Un

hospital entrena un modelo de deep learning para predecir la presencia de una enfermedad a partir de radiografías. Durante el preprocesamiento, las imágenes se guardan en carpetas diferentes según el diagnóstico real (por ejemplo, “positivo” y “negativo”). Al generar el dataset, el nombre de la carpeta o un identificador se incorpora accidentalmente como una variable o etiqueta auxiliar.

De esta manera, el modelo puede aprender a distinguir los patrones en los metadatos (la ruta de archivo o etiquetas implícitas) en lugar de la radiografía en sí. En validación interna el desempeño parece casi perfecto, pero en producción —donde las imágenes nuevas no incluyen esa información— el rendimiento cae abruptamente.

1.7.1 Contaminación train - test

Un tipo diferente de filtración ocurre cuando no se tiene cuidado de distinguir los datos de entrenamiento de los datos de validación/prueba. La validación debe ser una medida de cómo se desempeña el modelo en datos que no ha considerado previamente. Este proceso se podría dañar de maneras sutiles si los datos de validación afectan el comportamiento del preprocesamiento. A esto se le llama contaminación de entrenamiento-prueba.

Por ejemplo, supongamos que se ejecuta el preprocesamiento (como ajustar un imputador para valores faltantes) antes de llamar al método *train test split*. El modelo puede obtener buenos puntajes de validación, pero puede desempeñarse mal en la implementación. Se han incorporado datos de la validación o los datos de prueba en el entrenamiento. Si la ingeniería de características es más compleja, puede tener riesgos más elevados.

Si la validación se basa en una simple división entrenamiento-prueba, se debe excluir los datos de validación de cualquier tipo de ajuste, incluido el ajuste de pasos de preprocesamiento.

1.7.2 Prevención

Algunas pautas para la prevención de la fuga de datos pueden ser:

- Correcta división de train - test. Utilizar pipelines (por ejemplo, de scikit-learn) es una buena práctica para realizar los fits.
- Analizar si las características estarán disponibles en tiempo real. Si no lo están, removerlas.
- En caso de que las variables sean leaky pero no se pueda desprender de ellas (caso temperatura en consumo de demanda del ejemplo 3) analizar el error de predicción con el error de pronóstico.
- Añadir ruido a las variables, lo cual puede volver al algoritmo de aprendizaje más robusto y minimizar la fuga.

2 Modelos lineales para regresión

Los modelos lineales son algoritmos de aprendizaje que utilizan una función lineal entre una cierta cantidad de variables independientes x_i y una variable dependiente y . La cantidad de coeficientes es $n + 1$, donde n es la cantidad de variables independientes.

Según la cantidad de variables independientes, se puede pensar en este tipo de modelos como una figura geométrica lineal de dimensiones $n + 1$ que se ajuste con el menor error posible a los datos.

Es un modelo útil para describir relaciones simples y por lo tanto, es de muy alta interpretabilidad.

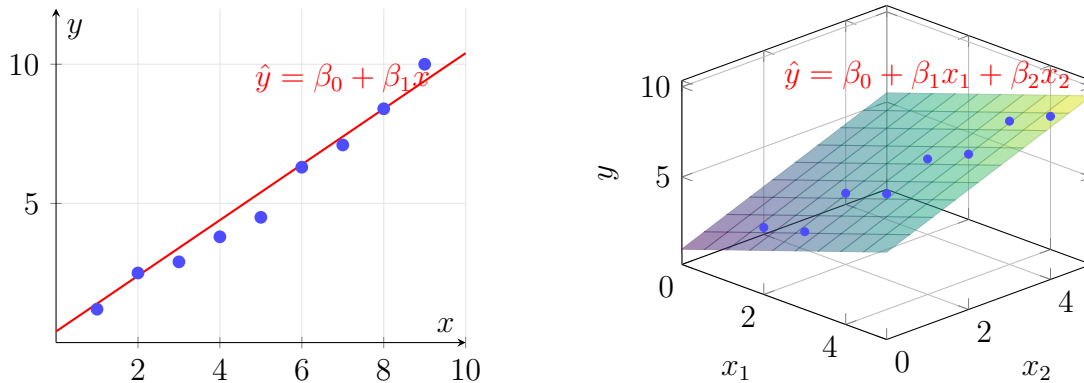


Figura 8: Modelos lineales: (arriba) regresión simple con una variable, (abajo) regresión múltiple con dos variables.

2.1 Regresión lineal simple

En este modelo, hay una sola variable independiente x , por lo que la figura geométrica resultante es una recta. El modelo se expresa como:

$$y = \beta_0 + \beta_1 x + \varepsilon$$

donde:

- β_0 es el intercepto (ordenada al origen),
- β_1 es la pendiente de la recta,
- ε representa el término de error aleatorio.

El desafío es encontrar los valores óptimos de los parámetros β_0 y β_1 que minimicen el error en las predicciones. Este problema se resuelve minimizando la **suma de los cuadrados de los residuos (RSS)**:

$$\text{RSS} = \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2$$

Este criterio lleva a las siguientes soluciones analíticas por **mínimos cuadrados ordinarios (OLS)**:

$$\frac{\partial \text{RSS}}{\partial \beta_0} = -2 \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i) = 0$$

$$\frac{\partial \text{RSS}}{\partial \beta_1} = -2 \sum_{i=1}^n x_i (y_i - \beta_0 - \beta_1 x_i) = 0$$

Resolviendo para β_0 :

$$\beta_0 = \bar{y} - \beta_1 \bar{x}$$

Resolviendo para β_1 :

$$\beta_1 = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2}$$

Notar que el punto $(\bar{x}, \bar{y})$ pertenece a la recta solución. De hecho, un cambio en las medias de x y de y cambian los coeficientes. Podemos concluir aquí que la presencia de outliers, que impactan significativamente en la media de los datos, van a impactar en la variabilidad de los coeficientes de la regresión lineal y consecuentemente, generarán varianza en el algoritmo.

¿Por qué se usa el error cuadrático? En primer lugar, usar una función de pérdida que sea directamente $(y_i - \hat{y}_i)$ parece intuitivamente correcta pero conlleva un gran error y es que si los errores se acumulan con signo, entonces pueden anularse entre sí. En otras palabras, dos errores de -10 y 10 sería lo mismo que tener dos errores de 0. Por otro lado, el cuadrado del error penaliza más fuertemente los errores grandes, lo cual ayuda a obtener un mejor ajuste global. Además, la función cuadrática es diferenciable y convexa, lo que permite una solución analítica cerrada. En cambio, minimizar el error absoluto requiere métodos de optimización más complejos, y puede generar soluciones no únicas. Se puede demostrar también que los RSS representan el modelo de máxima verosimilitud de la regresión lineal, aunque escapa los intereses de este curso [10]. Retomaremos, muy brevemente, esta idea cuando se discuta modelos lineales para clasificación.

2.2 Regresión lineal múltiple

La regresión lineal múltiple extiende la regresión simple al caso en que la variable dependiente y está influenciada por varias variables independientes $x_1, x_2, \dots, x_p$. El modelo se representa como:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p + \varepsilon$$

o en forma matricial:

$$\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon}$$

donde:

- $\mathbf{X}$ es la matriz de diseño que contiene las variables predictoras,
- $\boldsymbol{\beta}$ es el vector de parámetros,
- $\boldsymbol{\varepsilon}$ es el vector de errores.

Este modelo permite analizar el efecto conjunto de múltiples variables sobre y . La estimación de parámetros se realiza también mediante mínimos cuadrados.

La regresión múltiple es más flexible que la simple, pero también más susceptible a problemas como multicolinealidad.

La resolución de *OLS* da como resultado el vector de coeficientes:

$$\hat{\boldsymbol{\beta}} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

2.3 Métricas para regresión

2.3.1 Error cuadrático medio (Mean Squared Error o MSE)

Es la métrica más utilizada porque está directamente correlacionada con los *RSS*. Mide el promedio de los errores al cuadrado entre los valores reales y_i y las predicciones $\hat{y}_i$.

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Penaliza fuertemente los errores grandes debido a la elevación al cuadrado. Es útil cuando se quiere castigar más los errores significativos.

2.3.2 Raíz del error cuadrático medio (Root Mean Squared Error o RMSE)

Es simplemente la raíz cuadrada del MSE.

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

Tiene la ventaja de estar en las mismas unidades que la variable objetivo y , lo que facilita su interpretación directa, especialmente para comunicar resultados. Por ejemplo, si se quiere predecir el precio de una casa, el MSE puede dar 10.000 USD^2 mientras que el RMSE 100 USD .

2.3.3 Error absoluto medio (Mean Absolute Error o MAE)

Mide el promedio de los valores absolutos de los errores.

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

A diferencia del MSE, no penaliza en exceso los errores grandes, por lo que es más robusto ante outliers. Sin embargo, su función de error no es derivable en cero.

2.3.4 Error absoluto medio porcentual (Mean Absolute Percentage Error o MAPE)

Representa el error en términos relativos al valor real.

$$\text{MAPE} = \frac{100\%}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right|$$

Se expresa como un porcentaje, lo cual facilita la comparación entre modelos sobre distintas escalas. Por ejemplo, en el ejemplo de precios de casas, no es lo mismo hablar de un RMSE de 1.000 USD cuando las casas valen en promedio 10.000 USD (10% de error) o cuando valen 100.000 USD (1% de error).

No es adecuado cuando y_i puede tomar valores cercanos a cero.

2.3.5 R^2 Score

El coeficiente de determinación o R^2 mide la proporción de la varianza de la variable dependiente que es explicada por el modelo. Se define como:

$$R^2 = 1 - \frac{RSS}{TSS} = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

Donde TSS es la suma total de cuadrados. Por definición, el valor de R^2 varía entre $-\infty$ y 1.

Para pensar ¿qué significa que los valores de R^2 sean 1, 0.5, 0 o negativos?

2.3.6 R^2 ajustado

R^2 puede sesgarse por la cantidad de variables independientes del modelo. El **R^2 ajustado** corrige el R^2 teniendo en cuenta la cantidad de variables independientes utilizadas en el modelo, para evitar el falso positivo de mejora al agregar predictores irrelevantes:

$$R^2_{\text{adj}} = 1 - (1 - R^2) \cdot \frac{n - 1}{n - p - 1}$$

Donde n es la cantidad de observaciones y p es el número de predictores (excluyendo el intercepto).

Este valor solo aumenta si el nuevo predictor mejora el modelo más allá de lo que se esperaría por azar.

2.4 Gradiente descendiente

2.4.1 Motivación

El método de mínimos cuadrados ordinarios (OLS) provee una solución analítica a los coeficientes de un modelo lineal. Sin embargo, cuando trabajamos con grandes volúmenes de datos o funciones de costo más complejas, el cálculo exacto puede ser computacionalmente costoso o incluso inviable. En estos casos, se recurre a métodos numéricos como el

gradiente descendiente, que permiten encontrar una solución aproximada minimizando la función de costo de manera **iterativa**.

El gradiente de una función es un vector que apunta en la dirección de la mayor derivada de la función en un punto dado. Establece la dirección en la que la función está aumentando más rápidamente. Cuando se trata de minimizar una función de costo, se busca encontrar el punto donde la función alcanza su valor mínimo. En el mínimo de la función, el gradiente es igual a cero.

De esta manera, desplazarse en la dirección opuesta al gradiente, en teoría, podría ser una forma de acercarse al mínimo **global** de la función de pérdida, como se muestra en la figura 9. En las siguientes subsecciones se explicará por qué se usa la palabra "podría" en la oración anterior.

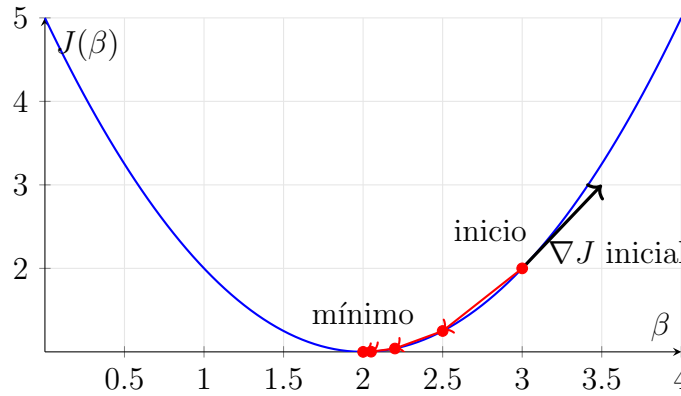


Figura 9: Gradiente descendiente: pasos hacia el mínimo de la función de costo $J(\beta)$ a partir de punto inicial, moviéndose en la dirección opuesta al vector gradiente.

2.4.2 Técnica del gradiente descendiente

Se parte de un vector inicial en el espacio de $n+1$ dimensiones (características + parámetro independiente) $v_{init} = (\beta_0, \beta_1, \dots, \beta_{n+1})$. Se obtiene el gradiente de la función de pérdida (u optimización) para ese punto en particular. La regla de actualización para un parámetro θ_j es:

$$\theta_j := \theta_j - \alpha \cdot \frac{\partial J(\theta)}{\partial \theta_j}$$

donde α es la tasa de aprendizaje (learning rate), $J(\theta)$ es la función de costo, típicamente el MSE en regresión, $\frac{\partial J(\theta)}{\partial \theta_j}$ es la derivada parcial de la función de costo respecto del parámetro θ_j .

En el caso del MSE:

$$\text{MSE}(\beta) = \frac{1}{m} \sum_{i=1}^m \left(y_i - \beta_0 - \sum_{j=1}^n \beta_j x_{ij} \right)^2$$

El gradiente del MSE con respecto a los parámetros es un vector que contiene las derivadas parciales:

$$\nabla_{\beta} \text{MSE} = \left[\frac{\partial \text{MSE}}{\partial \beta_0}, \frac{\partial \text{MSE}}{\partial \beta_1}, \dots, \frac{\partial \text{MSE}}{\partial \beta_n} \right]^T$$

En forma matricial, si definimos $\mathbf{X} \in R^{m \times (n+1)}$ como la matriz de diseño (donde la primera columna es de unos para el intercepto), e $\mathbf{y} \in R^m$ como el vector de etiquetas, entonces:

$$\text{MSE}(\beta) = \frac{1}{m} \|\mathbf{y} - \mathbf{X}\beta\|^2$$

Y su gradiente es:

$$\nabla_{\beta} \text{MSE} = -\frac{2}{m} \mathbf{X}^T (\mathbf{y} - \mathbf{X}\beta)$$

La actualización iterativa del gradiente descendente queda entonces:

$$\beta^{(t+1)} = \beta^{(t)} - \alpha \cdot \nabla_{\beta} \text{MSE}$$

2.4.3 Tasa de aprendizaje -learning rate-

La inicialización de los pesos β del algoritmo se hace de forma aleatoria, por lo que el único *hiperparámetro* que figura en las ecuaciones de la sección previa es la **tasa de aprendizaje** (learning rate o, usualmente en Python, 'lr'). Es decir, debe definirse su valor previo a ejecutar el algoritmo, aunque más adelante se discutirán algoritmos de tasa de aprendizaje variable.

La tasa de aprendizaje controla qué tan grandes son los pasos que da el algoritmo hacia el mínimo.

- Si α es muy grande, puede provocar que el algoritmo diverja.
- Si es muy pequeño, el algoritmo puede tardar mucho tiempo en converger o quedar atrapado en mínimos locales.

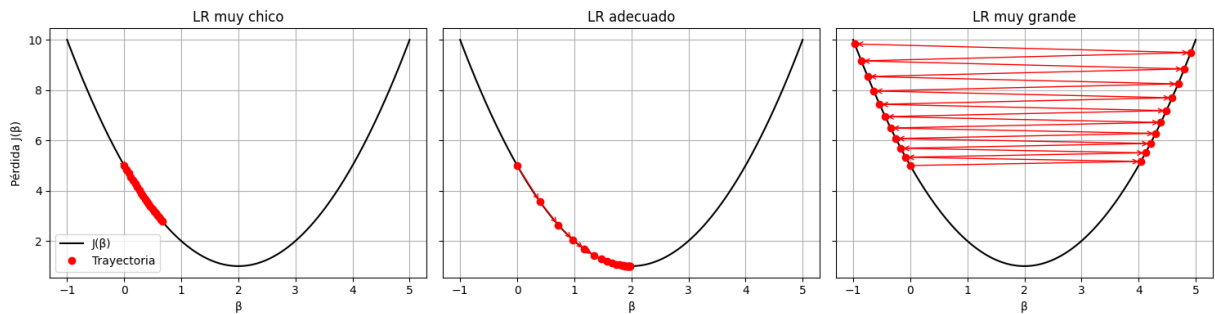


Figura 10: Impacto de la tasa de aprendizaje en la evolución del algoritmo hacia encontrar el mínimo.

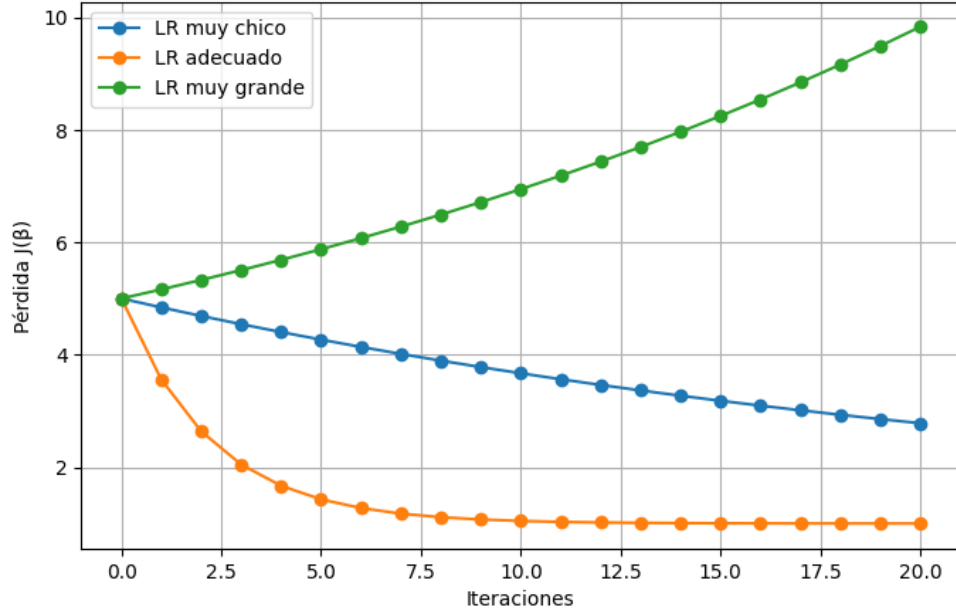


Figura 11: Impacto de la tasa de aprendizaje en la evolución de la pérdida vs n° de iteraciones.

2.4.4 Gradiente descendiente estocástico (SGD)

En lugar de calcular el gradiente completo sobre todos los datos, el **SGD** actualiza los parámetros usando un único ejemplo de entrenamiento $(x^{(i)}, y^{(i)})$.

$$\theta_j := \theta_j - \alpha \cdot \frac{\partial J(\theta; x^{(i)}, y^{(i)})}{\partial \theta_j}$$

Esto introduce una alta variabilidad en cada paso, lo cual puede ayudar a escapar de mínimos locales, pero también puede provocar oscilaciones.

2.4.5 Gradiente descendiente Mini-batch

Es una estrategia intermedia entre el GD completo y el SGD. Consiste en usar pequeños subconjuntos (mini-batches) de datos en cada iteración para calcular el gradiente.

$$\theta_j := \theta_j - \alpha \cdot \frac{1}{m} \sum_{i \in \text{batch}} \frac{\partial J(\theta; x^{(i)}, y^{(i)})}{\partial \theta_j}$$

Permite aprovechar la paralelización y obtener una estimación más estable del gradiente que el SGD.

2.4.6 Comparación GD, SGD, mini-batch GD

- **Gradient Descent (GD)**: utiliza todo el conjunto de datos para cada actualización. Es preciso pero costoso para datasets grandes, ya que mantiene todos los datos en memoria.

- **Stochastic Gradient Descent (SGD)**: utiliza un solo ejemplo por actualización. Es de convergencia rápida y permite exploración, pero es inestable y presenta ruido. Termina siendo un caso extremo de Mini-batch, con batch size = largo del dataset.
- **Mini-batch Gradient Descent**: aprovecha paralelismo y reduce la varianza del gradiente, siendo la opción más común en la práctica, con el equilibrio óptimo entre rapidez y ruido.

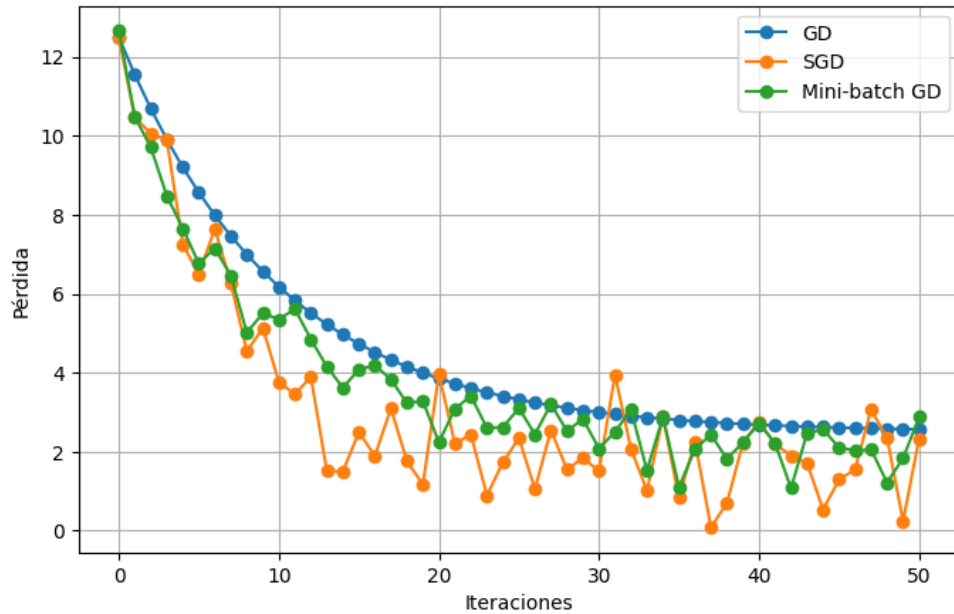


Figura 12: Evolución de la pérdida vs n° de iteraciones para los tres tipos de gradiente descendiente.

2.5 Regularización

2.5.1 Motivación: reducción de la varianza en un modelo de regresión lineal

La regresión lineal puede tener un muy buen ajuste sobre los datos de entrenamiento, pero si el modelo es muy flexible (por ejemplo, cuando hay muchas variables explicativas o colinealidad), es probable que tenga alta varianza y un mal rendimiento sobre datos no vistos. Este es un caso típico de sobreajuste (overfitting).

La regularización consiste en agregar una penalización al costo que impide que los coeficientes crezcan demasiado, lo que favorece soluciones más estables y robustas, y reduce la varianza del modelo, ya que quita esa flexibilidad del algoritmo inicial. Como desventaja, se obtendrá un modelo que se desempeñe de forma más pobre en entrenamiento.

2.5.2 Regularización L2: Ridge

La regularización Ridge (también conocida como Tikhonov regularization) modifica la función de pérdida de la regresión lineal agregando una penalización proporcional al cuadrado de la norma L2 del vector de coeficientes β :

$$\text{RSS}_{\text{Ridge}} = \sum_{i=1}^n (y_i - \mathbf{x}_i^\top \boldsymbol{\beta})^2 + \lambda \sum_{j=1}^p \beta_j^2$$

donde $\lambda \geq 0$ es el hiperparámetro de regularización que controla la penalización.

Si $\lambda = 0$, recuperamos la regresión lineal clásica. Si $\lambda \rightarrow \infty$, los coeficientes tienden a cero, por lo que el modelo tiende a ser simplemente un término constante (y por optimización, será la media de los datos de entrenamiento).

La solución analítica en forma matricial es:

$$\hat{\boldsymbol{\beta}}^{\text{Ridge}} = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}$$

2.5.3 Regularización L1: Lasso

La regularización Lasso (Least Absolute Shrinkage and Selection Operator) penaliza la suma de los valores absolutos de los coeficientes, es decir, la norma L1:

$$\text{RSS}_{\text{Lasso}} = \sum_{i=1}^n (y_i - \mathbf{x}_i^\top \boldsymbol{\beta})^2 + \lambda \sum_{j=1}^p |\beta_j|$$

Este tipo de regularización tiende a forzar algunos coeficientes exactamente a cero, lo que realiza una selección de variables implícita. Sin embargo, a diferencia de Ridge, no tiene una solución analítica cerrada y debe resolverse mediante algoritmos como el método del subgradiente o coordinate descent.

2.5.4 L1 vs L2

Ambas regularizaciones L1 (Lasso) y L2 (Ridge) comparten el objetivo de penalizar los modelos complejos, pero lo hacen de manera diferente, lo que produce comportamientos distintos sobre los coeficientes.

Una forma intuitiva de comprender sus diferencias es desde una perspectiva geométrica. Consideremos el problema de minimizar la función de pérdida (por ejemplo, el error cuadrático medio) sujeta a una restricción sobre la magnitud de los coeficientes.

- En Ridge (L2), la penalización se basa en la norma euclidiana de los coeficientes, es decir, $\lambda \sum_{j=1}^p \beta_j^2$. Esto impone una restricción del tipo:

$$\sum_{j=1}^p \beta_j^2 \leq t$$

que corresponde a una esfera (o círculo en 2D).

- En Lasso (L1), la penalización se basa en la suma de los valores absolutos de los coeficientes, es decir, $\lambda \sum_{j=1}^p |\beta_j|$. Esto impone una restricción del tipo:

$$\sum_{j=1}^p |\beta_j| \leq t$$

que corresponde a un rombo (en 2D) o un octaedro (en dimensiones más altas).

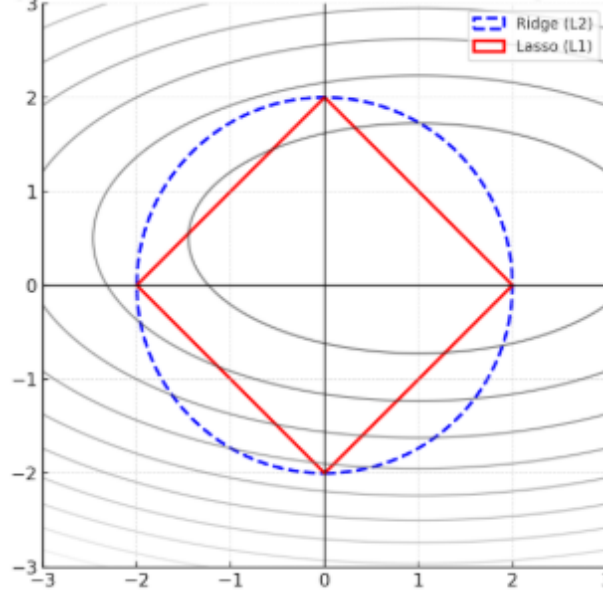


Figura 13: Curvas de nivel de función de pérdida original y restricción LASSO vs Ridge.

La diferencia en la forma de estas regiones de restricción tiene un efecto importante. El mínimo de la función de pérdida regularizada se encuentra en el punto donde las curvas de nivel del error intersectan la región de restricción.

Dado que el rombo tiene vértices en los ejes, es más probable que el punto de intersección ocurra en uno de esos vértices, lo que implica que uno o más coeficientes sean exactamente cero. Esto explica por qué **LASSO tiende a generar modelos más esparsos** (con menos variables seleccionadas), mientras que **Ridge tiende a reducir la magnitud de todos los coeficientes pero no llevarlos exactamente a cero**.

Por esta razón, LASSO puede usarse también como una técnica de selección de variables, mientras que Ridge se enfoca más en reducir la varianza del modelo.

Si se sospecha que sólo unas pocas variables son relevantes, LASSO puede ser más efectivo. Si todas las variables tienen algún grado de influencia, Ridge puede ser preferible.

2.5.5 Generalización

Las técnicas de regularización L1 (Lasso) y L2 (Ridge) pueden verse como casos particulares de una familia más general de regularizaciones basada en las llamadas normas L^p , también conocidas como ℓ_p -normas. Dada una solución de regresión lineal con vector de coeficientes $\beta \in R^p$, se define la norma L^p como:

$$\|\beta\|_p = \left(\sum_{j=1}^p |\beta_j|^p \right)^{1/p}$$

La regularización basada en una norma L^p consiste en resolver el siguiente problema de minimización penalizada:

$$\min_{\boldsymbol{\beta}} \left\{ \sum_{i=1}^n (y_i - \mathbf{x}_i^\top \boldsymbol{\beta})^2 + \lambda \|\boldsymbol{\beta}\|_p^p \right\}$$

Los casos particulares más conocidos son:

- $p = 2$: Ridge Regression.
- $p = 1$: Lasso Regression.
- $0 < p < 1$: se promueve aún más la esparsidad, pero el problema se vuelve no convexa y más difícil de optimizar.
- $p > 2$: castigan más severamente los coeficientes grandes, pero no tienden a inducir esparsidad.

Esta formulación permite explorar un espectro continuo de regularizaciones entre L1 y L2, ajustando el parámetro p para adaptarse a diferentes estructuras de datos y objetivos de modelado.

Desde un punto de vista geométrico, cada valor de p define una "bola" diferente en el espacio de parámetros (figura 14):

- Para $p = 1$, es un rombo (en 2D), con esquinas alineadas con los ejes.
- Para $p = 2$, es un círculo.
- Para $p < 1$, las bolas tienen puntas más afiladas, incentivando soluciones más esparsas.
- Para $p > 2$, a medida que más grande p , la bola tiende a un cuadrado con lados paralelos a los ejes.

Este enfoque más general permite diseñar penalizaciones personalizadas, dependiendo del grado de esparsidad deseado, la convexidad del problema, y la sensibilidad a los coeficientes grandes.

2.5.6 ElasticNet

ElasticNet combina ambas penalizaciones L1 y L2 en una única función de pérdida:

$$\text{RSS}_{\text{ElasticNet}} = \sum_{i=1}^n (y_i - \mathbf{x}_i^\top \boldsymbol{\beta})^2 + \lambda_1 \sum_{j=1}^p |\beta_j| + \lambda_2 \sum_{j=1}^p \beta_j^2$$

Esta técnica combina las ventajas de Lasso y Ridge: logra realizar selección de variables al mismo tiempo que evita algunos de los problemas de Lasso (como la selección aleatoria entre variables correlacionadas).

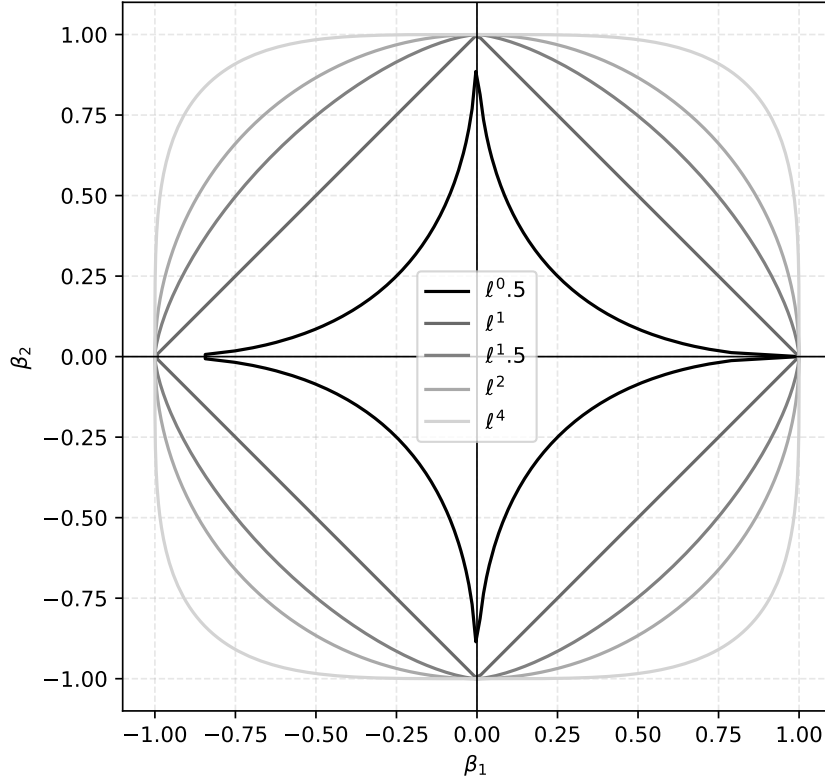


Figura 14: Bolas para distintos valores de p .

3 Modelos lineales para clasificación

Ya se ha definido un problema de clasificación en 1.3.2, sin embargo es importante recordar cuál es el objetivo de resolver un problema de esta índole: determinar un algoritmo de aprendizaje capaz de recibir una entrada de diversos predictores para producir una salida categórica. Por ejemplo, si se quiere determinar si el día de mañana va a llover o no. La salida es claramente binaria. Sin embargo, hay información adicional que es de gran utilidad, tanto para el consumidor de esta output como para luego mejorar la explicabilidad del modelo, y consiste en responder a la pregunta: *¿cuál es la **probabilidad** que el día de mañana llueva?*. Por ello, en un modelo de clasificación, el objetivo primario será estimar la función de probabilidad, para luego aplicar una función adicional y convertirlo en la salida categórica. Ya que la probabilidad de pertenencia a una clase es un valor continuo entre 0 y 1, podría pensarse como un problema de regresión. El gran desafío, respecto de los problemas de la unidad anterior, es que no se cuenta con el valor real de probabilidad de una observación, sino con su clase asociada.

3.1 Clasificación binaria vs multiclase

En la clasificación binaria, el objetivo es asignar una de dos clases posibles, como $[0, 1]$ o [Sí, No], a una observación. Por ejemplo, predecir si un correo es spam o no lo es.

En la clasificación multiclase, el número de categorías posibles es mayor a dos. Por

ejemplo, predecir el tipo de flor (setosa, versicolor, virginica) según sus características.

Muchas técnicas de clasificación binaria pueden extenderse al caso multiclase mediante estrategias como:

- **One-vs-All (OvA):** se entrena un clasificador por cada clase contra el resto.
- **One-vs-One (OvO):** se entrena un clasificador por cada par de clases.
- **Multinomial:** se adapta directamente la función de costo y activación para múltiples clases (ej., regresión logística softmax).

3.2 La regresión lineal como clasificador

Suponiendo un problema con dos clases de salida $y \in \{0, 1\}$, matemáticamente es totalmente posible entrenar una regresión lineal. El resultado será una recta (en caso de tener una sola variable explicativa), con la forma:

$$\hat{y} = \mathbf{x}^\top \boldsymbol{\beta}$$

y luego tomar una decisión de clasificación utilizando un umbral. Por ejemplo:

$$\text{Se clasifica como } \begin{cases} 1 & \text{si } \hat{y} \geq 0.5 \\ 0 & \text{si } \hat{y} < 0.5 \end{cases}$$

El modelo de regresión lineal produce predicciones $\hat{y}$ en todo el eje real, es decir, $\hat{y} \in (-\infty, \infty)$, mientras que las etiquetas verdaderas pertenecen al conjunto $\{0, 1\}$. Esto genera varios problemas:

- **Interpretabilidad:** No se puede interpretar el valor de $\hat{y}$ como una probabilidad.
- **Robustez:** Valores atípicos o errores de predicción pueden dar lugar a salidas muy alejadas de $[0, 1]$.
- **Mal ajuste:** La función de pérdida utilizada (error cuadrático medio) no está pensada para clasificación.
- **Sólo posible para problemas binarios:** la resolución de este problema requirió una codificación ordinal, que para tres o más clases puede dejar de tener sentido.

Para resolver estos problemas, se requiere, en primera instancia, una función que restrinja la salida del modelo al intervalo $[0, 1]$. La solución es utilizar una transformación no lineal conocida como **función logística** o **sigmoide**:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

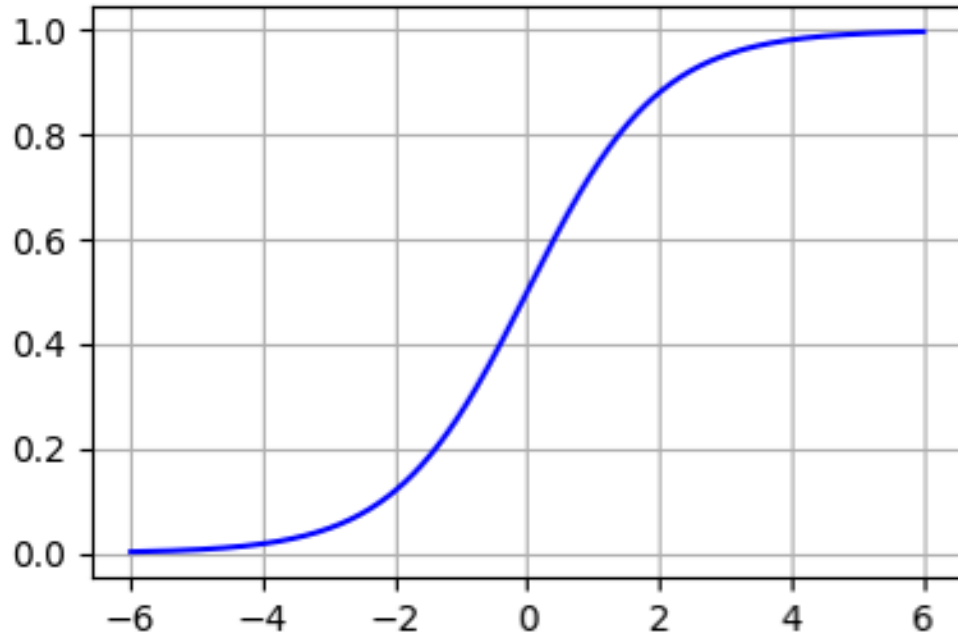


Figura 15: Función sigmoidea.

donde $z = \mathbf{x}^\top \boldsymbol{\beta}$. De esta forma, el nuevo modelo se convierte en:

$$\hat{y} = \sigma(\mathbf{x}^\top \boldsymbol{\beta})$$

Ahora, $\hat{y}$ puede interpretarse como una probabilidad, y la clasificación se realiza de la siguiente manera:

$$\text{Clasificamos como } \begin{cases} 1 & \text{si } \hat{y} \geq \textit{umbral} \\ 0 & \text{si } \hat{y} < 1 - \textit{umbral} \end{cases}$$

3.3 Regresión logística

El razonamiento anterior deriva en la **regresión logística**, que modela la probabilidad condicional de la clase 1 (de un problema binario) como una función sigmoide del predictor lineal. La función logística se define como:

$$P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{x}^\top \boldsymbol{\beta}) = \frac{1}{1 + e^{-\mathbf{x}^\top \boldsymbol{\beta}}}$$

Produce probabilidades entre 0 y 1 y es una función continua y diferenciable, lo que facilita su entrenamiento.

3.4 Chances (odds)

Las **chances** (u *odds*) se definen como la razón entre la probabilidad de que ocurra un evento y la probabilidad de que no ocurra:

$$\text{Odds} = \frac{P(y = 1 \mid \mathbf{x})}{P(y = 0 \mid \mathbf{x})} = \frac{P(y = 1 \mid \mathbf{x})}{1 - P(y = 1 \mid \mathbf{x})}$$

Por ejemplo, si la probabilidad de que un paciente tenga una enfermedad es 0.8, entonces las chances son:

$$\frac{0.8}{1 - 0.8} = \frac{0.8}{0.2} = 4$$

Lo que se interpreta como “4 a 1 a favor de que tenga la enfermedad”.

3.5 Logaritmo de las chances (log-odds o logit)

El **logaritmo de las chances**, conocido como *logit*, es la función de enlace en la regresión logística:

$$\text{logit}(p) = \log\left(\frac{p}{1 - p}\right)$$

Y en nuestro caso se modela como:

$$\text{logit}(P(y = 1 \mid \mathbf{x})) = \mathbf{x}^\top \boldsymbol{\beta}$$

Esta transformación convierte la probabilidad (acotada entre 0 y 1) en una variable no acotada, lo cual permite modelarla mediante una combinación lineal de las variables.

Frontera de decisión lineal Para tomar una decisión de clasificación, se elige un **umbral de probabilidad**, típica pero no necesariamente 0.5. Es decir:

$$\hat{y} = \begin{cases} 1 & \text{si } P(y = 1 \mid \mathbf{x}) \geq 0.5 \\ 0 & \text{si } P(y = 1 \mid \mathbf{x}) < 0.5 \end{cases}$$

Como $P(y = 1 \mid \mathbf{x}) = \frac{1}{1 + e^{-\mathbf{x}^\top \boldsymbol{\beta}}}$, si igualamos esta expresión al umbral 0.5, obtenemos:

$$\frac{1}{1 + e^{-\mathbf{x}^\top \boldsymbol{\beta}}} = 0.5 \quad \Rightarrow \quad \mathbf{x}^\top \boldsymbol{\beta} = 0$$

Este valor define la **frontera de decisión**: el hiperplano $\mathbf{x}^\top \boldsymbol{\beta} = 0$ separa el espacio de las variables en dos regiones:

- Si $\mathbf{x}^\top \boldsymbol{\beta} > 0$, entonces $P(y = 1 \mid \mathbf{x}) > 0.5 \Rightarrow \hat{y} = 1$.
- Si $\mathbf{x}^\top \boldsymbol{\beta} < 0$, entonces $P(y = 1 \mid \mathbf{x}) < 0.5 \Rightarrow \hat{y} = 0$.

Por lo tanto, aunque el modelo predice probabilidades, la **decisión final es lineal**, y la frontera entre clases es un plano (o recta en 2D).

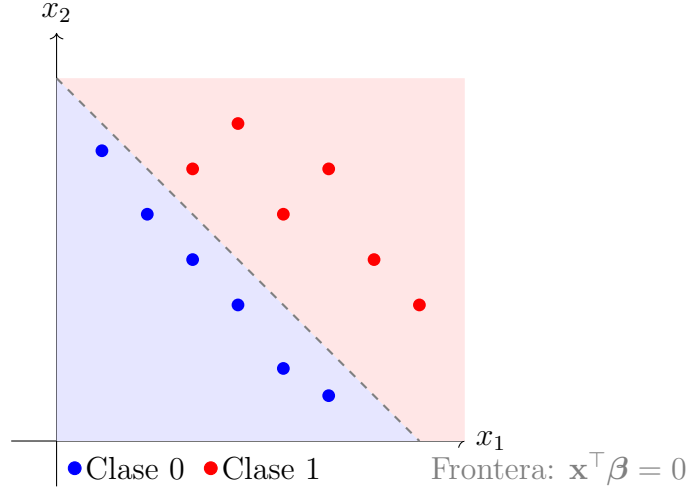


Figura 16: Visualización de la frontera de decisión lineal en regresión logística.

3.6 ¿Por qué se llama regresión logística?

Se ha modelado $\text{logit}(p)$ como una regresión lineal múltiple. De aquí su nombre: es una **regresión**, pero aplicada a los **logit(p)**. La gran diferencia con una regresión lineal ordinaria, es que $\text{logit}(p)$ en los datos de entrenamiento no es conocido, por lo que la función de pérdida RSS no será de ayuda para optimizar los coeficientes.

3.7 Estimación de los coeficientes de la regresión logística

En lugar de minimizar el error cuadrático como en la regresión lineal, en la regresión logística se adopta un enfoque de **máxima verosimilitud**. El objetivo es encontrar los parámetros β que maximizan la probabilidad de observar los datos disponibles [10].

3.7.1 Función de costo por máxima verosimilitud

Dado un conjunto de datos de entrenamiento $\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^n$, donde $y^{(i)} \in \{0, 1\}$ representa la clase, y $\mathbf{x}^{(i)}$ es el vector de predictores, la función de probabilidad de una sola observación se modela como:

$$P(y^{(i)} = 1 \mid \mathbf{x}^{(i)}; \beta) = \sigma(\beta^T \mathbf{x}^{(i)})$$

donde $\sigma(z) = \frac{1}{1+e^{-z}}$ es la función logística o sigmoide.

La función de verosimilitud total es:

$$\mathcal{L}(\beta) = \prod_{i=1}^n [\sigma(\beta^T \mathbf{x}^{(i)})]^{y^{(i)}} [1 - \sigma(\beta^T \mathbf{x}^{(i)})]^{1-y^{(i)}}$$

Trabajar con el logaritmo de la verosimilitud (log-verosimilitud) simplifica el cálculo:

$$\log \mathcal{L}(\boldsymbol{\beta}) = \sum_{i=1}^n [y^{(i)} \log \sigma(\boldsymbol{\beta}^T \mathbf{x}^{(i)}) + (1 - y^{(i)}) \log(1 - \sigma(\boldsymbol{\beta}^T \mathbf{x}^{(i)}))]$$

El objetivo es maximizar esta función, pero ya que minimizar es más simple computacionalmente, se buscará minimizar su opuesto.

3.7.2 Entropía binaria cruzada

Minimizar la **función de costo** en regresión logística equivale a minimizar la **entropía cruzada** (o *log loss*), definida como:

$$J(\boldsymbol{\beta}) = -\frac{1}{n} \sum_{i=1}^n [y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})]$$

donde $\hat{y}^{(i)} = \sigma(\boldsymbol{\beta}^T \mathbf{x}^{(i)})$ es la probabilidad predicha de la clase positiva.

Esta función mide la discrepancia entre la etiqueta real $y \in \{0, 1\}$ y la probabilidad predicha por un modelo $\hat{y} \in (0, 1)$.

- Si $y = 1$, la función se convierte en:

$$\mathcal{L}(1, \hat{y}) = -\log(\hat{y})$$

La pérdida crece mucho cuando $\hat{y} \rightarrow 0$. Es decir, si el modelo predice una baja probabilidad de clase 1 cuando en realidad es clase 1, la penalización es grande, como se muestra en la figura 17.

- De forma análoga, si $y = 0$, la función se convierte en:

$$\mathcal{L}(0, \hat{y}) = -\log(1 - \hat{y})$$

La pérdida crece mucho cuando $\hat{y} \rightarrow 1$. Es decir, se penaliza que el modelo esté muy seguro de que es clase 1 cuando en realidad es clase 0

3.7.3 Regularización

Al igual que en regresión lineal, puede aplicarse **regularización** para reducir la varianza y evitar el sobreajuste. Con Ridge (L2) se añade un término $\lambda \|\boldsymbol{\beta}\|_2^2$ a la función de costo. y con Lasso (L1) se añade un término $\lambda \|\boldsymbol{\beta}\|_1$.

El costo regularizado se convierte en:

$$J_{reg}(\boldsymbol{\beta}) = J(\boldsymbol{\beta}) + \lambda R(\boldsymbol{\beta})$$

donde $R(\boldsymbol{\beta})$ puede ser la norma L1 o L2.

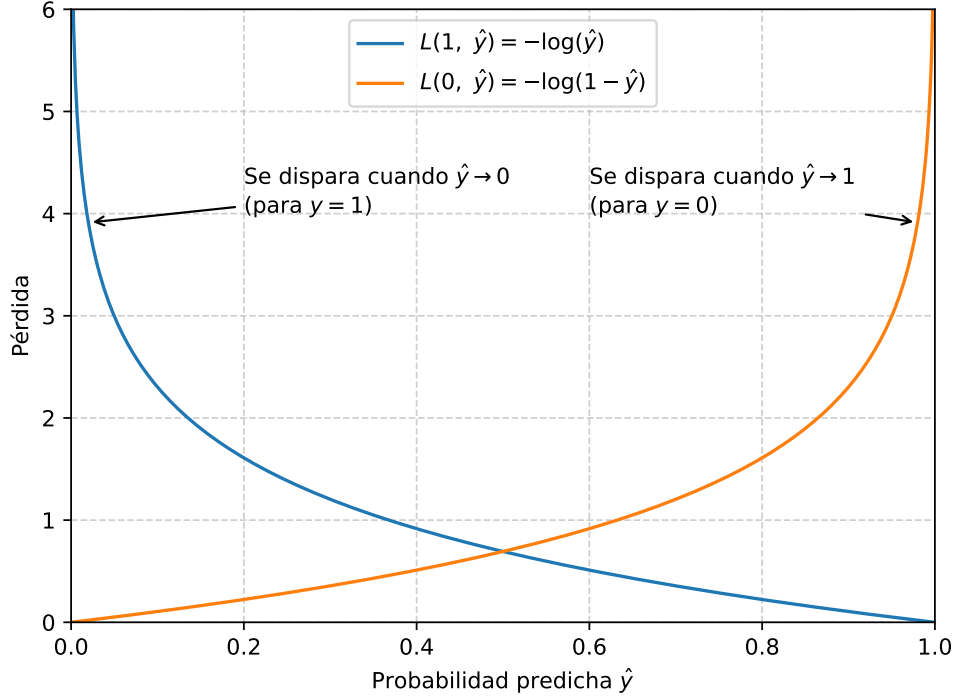


Figura 17: Entropía cruzada binaria (log loss) para $y = 1$ y $y = 0$.

3.7.4 Estimación por gradiente descendiente

Dado que no existe una solución cerrada como en la regresión lineal, se utiliza el **gradiente descendiente** para minimizar la función de costo. El gradiente de la entropía cruzada respecto a los coeficientes β es:

$$\nabla J(\beta) = \frac{1}{n} \sum_{i=1}^n (\hat{y}^{(i)} - y^{(i)}) \mathbf{x}^{(i)}$$

Por linealidad de la derivada y suma,

$$\frac{\partial J(\theta)}{\partial \theta_j} = -\frac{1}{m} \sum_{i=1}^m \frac{\partial}{\partial \theta_j} [y^{(i)} \log h_{\theta}(x^{(i)}) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)}))]$$

Se define, para abreviar:

$$h^{(i)} \equiv h_{\theta}(x^{(i)}) = g(\theta^T x^{(i)})$$

La derivada interior queda:

$$\frac{\partial}{\partial \theta_j} [y^{(i)} \log h^{(i)} + (1 - y^{(i)}) \log(1 - h^{(i)})] = y^{(i)} \frac{1}{h^{(i)}} \frac{\partial h^{(i)}}{\partial \theta_j} + (1 - y^{(i)}) \frac{1}{1 - h^{(i)}} \frac{\partial(1 - h^{(i)})}{\partial \theta_j}$$

Notar que:

$$\frac{\partial(1 - h^{(i)})}{\partial \theta_j} = -\frac{\partial h^{(i)}}{\partial \theta_j}$$

Por lo tanto,

$$= y^{(i)} \frac{1}{h^{(i)}} \frac{\partial h^{(i)}}{\partial \theta_j} - (1 - y^{(i)}) \frac{1}{1 - h^{(i)}} \frac{\partial h^{(i)}}{\partial \theta_j} = \left(\frac{y^{(i)}}{h^{(i)}} - \frac{1 - y^{(i)}}{1 - h^{(i)}} \right) \frac{\partial h^{(i)}}{\partial \theta_j}$$

Ahora, se calcula $\frac{\partial h^{(i)}}{\partial \theta_j}$.

Recordemos que $h^{(i)} = g(z^{(i)})$, con $z^{(i)} = \theta^T x^{(i)}$.

La derivada de la sigmoide es:

$$g'(z) = g(z)(1 - g(z))$$

Por lo tanto,

$$\frac{\partial h^{(i)}}{\partial \theta_j} = g'(z^{(i)}) \frac{\partial z^{(i)}}{\partial \theta_j} = h^{(i)}(1 - h^{(i)})x_j^{(i)}$$

Se sustituye en la expresión anterior:

$$\left(\frac{y^{(i)}}{h^{(i)}} - \frac{1 - y^{(i)}}{1 - h^{(i)}} \right) h^{(i)}(1 - h^{(i)})x_j^{(i)}$$

Simplificamos la expresión entre paréntesis multiplicado por $h^{(i)}(1 - h^{(i)})$:

$$= (y^{(i)}(1 - h^{(i)}) - (1 - y^{(i)})h^{(i)})x_j^{(i)} = (y^{(i)} - y^{(i)}h^{(i)} - h^{(i)} + y^{(i)}h^{(i)})x_j^{(i)} = (y^{(i)} - h^{(i)})x_j^{(i)}$$

—

Finalmente, el gradiente queda:

$$\boxed{\frac{\partial J(\theta)}{\partial \theta_j} = -\frac{1}{m} \sum_{i=1}^m (y^{(i)} - h_{\theta}(x^{(i)}))x_j^{(i)} = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})x_j^{(i)}}$$

Este gradiente se usa para actualizar los coeficientes iterativamente:

$$\beta^{(t+1)} = \beta^{(t)} - \alpha \cdot \nabla J(\beta^{(t)})$$

donde α es el *learning rate*.

3.8 Problemas multiclase

En problemas de clasificación multiclase, el objetivo es asignar una observación de entrada x a una de K clases posibles, donde $K > 2$. A diferencia de la regresión logística binaria, que modela la probabilidad de una sola clase, en el caso multiclase se desea estimar simultáneamente la probabilidad de pertenencia a cada clase. Para ello, se utiliza la función **softmax** como generalización de la función sigmoide.

Función softmax

Dado un vector de puntuaciones (también llamadas logits) $z = (z_1, z_2, \dots, z_K)$, la función softmax transforma estos valores en una distribución de probabilidad sobre las clases:

$$\text{softmax}(z)_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}} \quad \text{para } k = 1, 2, \dots, K$$

Cada valor $\text{softmax}(z)_k$ representa la probabilidad predicha de que la observación pertenezca a la clase k , cumpliendo que:

$$0 \leq \text{softmax}(z)_k \leq 1, \quad \sum_{k=1}^K \text{softmax}(z)_k = 1$$

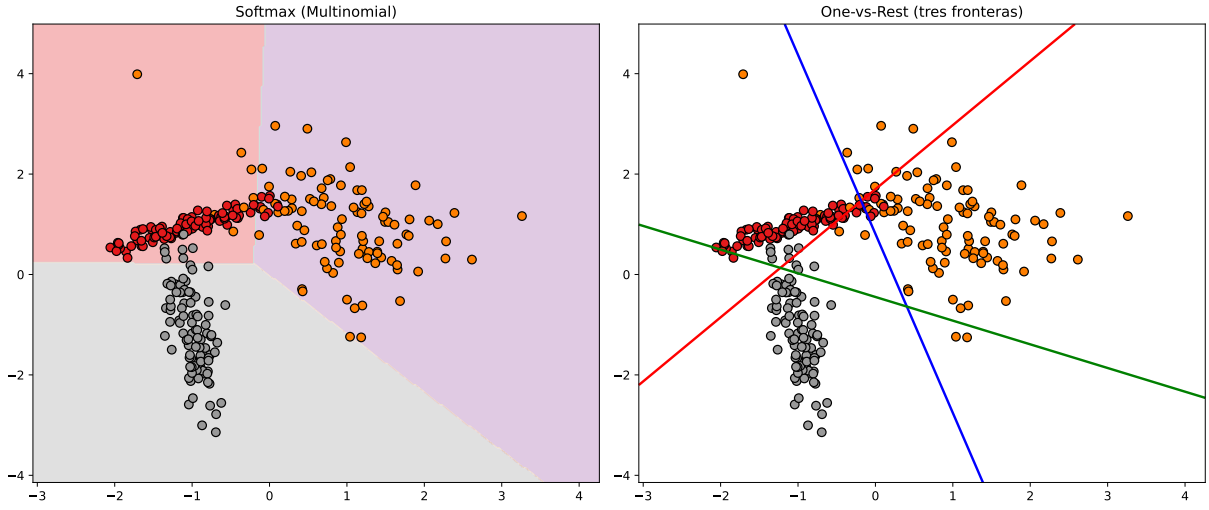


Figura 18: Clasificación multiclase: a la izquierda función softmax y a la derecha One-vs-Rest.

Función de costo: entropía cruzada categórica

Para entrenar el modelo, se emplea una función de pérdida que compare la distribución predicha por el modelo con la verdadera clase (codificada como un vector one-hot). La función más común es la **entropía cruzada categórica**, definida como:

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^K y_k^{(i)} \log \hat{y}_k^{(i)}$$

donde:

- m : número de ejemplos de entrenamiento,
- $y_k^{(i)}$: valor real (0 o 1) para la clase k en el ejemplo i ,

- $\hat{y}_k^{(i)}$: probabilidad predicha por el modelo para la clase k en el ejemplo i .

El modelo se entrena ajustando los parámetros θ para minimizar la función de pérdida $J(\theta)$, con métodos primos de gradiente descendiente.

4 Métricas para clasificación

Como ya se ha discutido, la salida del modelo es una función de probabilidad pero no se tienen valores reales de la misma. La única forma de hacer una comparación y poder efectivamente medir el desempeño del modelo es usar las etiquetas reales de clases. En clasificación de n clases, hay n posibilidades para la etiqueta real y n posibilidades como output del modelo. De aquí que, en una matriz $n \times n$, se puede representar todas las combinaciones posibles. Esta matriz se llama *matriz de confusión*.

4.1 Matriz de confusión

La matriz de confusión permite visualizar el número de predicciones correctas e incorrectas realizadas por el modelo, separadas por clase.

En un problema de clasificación binaria, la matriz de confusión tiene la siguiente forma:

Clase real	Positiva	20105	10190
	Negativa	5496	15753
		Positiva	Negativa
		Clase predicha	

Figura 19: Matriz de confusión.

Donde:

- **TP** (True Positives): verdaderos positivos, casos positivos correctamente clasificados.

- **FN** (False Negatives): falsos negativos, casos positivos clasificados incorrectamente como negativos. Por ejemplo, no detectar una enfermedad en un paciente que realmente la tiene.
- **FP** (False Positives): falsos positivos, casos negativos clasificados incorrectamente como positivos. Por ejemplo, diagnosticar erróneamente una enfermedad en una persona sana.
- **TN** (True Negatives): verdaderos negativos, casos negativos correctamente clasificados.

La utilidad de la matriz de confusión radica en que permite identificar no sólo cuántas veces el modelo acierta, sino también *cómo se equivoca*. El impacto de cometer un FP o un FN depende del contexto. Por ejemplo, en medicina, los falsos negativos suelen ser más graves, ya que implican no tratar a un paciente enfermo. En cambio, en un filtro antispam, un falso positivo (marcar como spam un correo legítimo) puede ser más inconveniente que un falso negativo.

Esta matriz se puede extender fácilmente a problemas multiclase, donde se representa en forma de una matriz cuadrada de tamaño $K \times K$, con K siendo el número de clases.

Si bien cada uno de estos n^2 componentes de la matriz podría utilizarse como métrica, lo más común es toparse con combinaciones de ellos, como se verá en las siguientes secciones.

4.2 Accuracy

La métrica más básica derivada de la matriz de confusión es el accuracy (exactitud), que representa la proporción de predicciones correctas sobre el total de predicciones:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Puede ser una métrica útil cuando las clases están balanceadas. Sin embargo, puede ser engañosa en contextos con clases desbalanceadas. Por ejemplo, si el 95% de los datos pertenecen a la clase negativa, un modelo que siempre predice "negativo" tendrá una accuracy del 95% sin ser realmente útil. Además, el peso que se le da a FP y a FN es el mismo, por lo que si uno de ellos tiene un impacto superior al otro, no se refleja en la métrica.

4.3 Recall (sensibilidad o tasa de verdaderos positivos)

Mide la capacidad del modelo para identificar correctamente los casos positivos. Se define como:

$$\text{Recall} = \frac{TP}{TP + FN}$$

Indica qué proporción de los casos realmente positivos fueron clasificados como tal. Un valor bajo de recall implica que el modelo está dejando pasar muchos verdaderos positivos (es decir, comete muchos falsos negativos).

Notar que el recall se calcula con datos de una sola clase, por lo que pueden calcularse dos recalls distintos: el de la clase positiva y el de la clase negativa.

Esta métrica puede ser importante en contextos donde los falsos negativos son costosos o peligrosos. Por ejemplo, en diagnósticos médicos, detectar una enfermedad cuando realmente está presente es fundamental. Un modelo con bajo recall podría pasar por alto muchos casos positivos, resultando en consecuencias graves.

Sin embargo, un modelo que maximiza el recall puede hacerlo a expensas de un mayor número de falsos positivos, ya que puede etiquetar muchos casos como positivos para no dejar pasar ninguno. Es decir, un modelo que siempre predice la clase positiva tiene el recall máximo de 1. Por eso, es importante no analizar el recall de forma aislada, sino como parte de una métrica más elaborada.

En problemas multiclase, se puede calcular un recall distinto para cada clase tratándola como la clase positiva y agrupando el resto como clase negativa.

4.4 Precision (precisión)

Mide la proporción de predicciones positivas que realmente son correctas. Se calcula como:

$$\text{Precision} = \frac{TP}{TP + FP}$$

Un valor bajo de precision indica que el modelo está generando muchos falsos positivos, es decir, que muchas de sus predicciones positivas son incorrectas.

Esta métrica puede ser utilizada en contextos donde los falsos positivos tienen un alto costo. Por ejemplo, en un sistema de detección de spam, un falso positivo implicaría que un correo legítimo se marque erróneamente como spam. Si la precisión es baja, muchas de las alertas positivas serán incorrectas, lo que puede generar pérdida de confianza en el sistema.

Sin embargo, y de forma muy análoga con el recall, un modelo que maximiza la precisión puede ser conservador a la hora de predecir positivos, lo que podría derivar en un bajo recall. Por ello, al igual que el recall, la precisión no debe evaluarse de forma aislada.

En tareas multiclase, se puede calcular la precisión para cada clase individual, y luego promediar según diferentes esquemas (macro, micro, weighted).

4.5 F1-score

Es la media armónica entre precision y recall, y se utiliza cuando se desea un equilibrio entre ambas métricas:

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Esta métrica penaliza fuertemente los casos en los que una de las dos (precision o recall) es muy baja, a diferencia de lo que haría el promedio entre ellas.

Si sustituimos precisión y recall de la ecuación anterior,

$$\text{F1-score} = \frac{2TP}{2TP + FP + FN}$$

Al incorporar ambos falsos en el denominador, suele ser más utilizado para problemas de clases desbalanceadas. Notar igualmente que la importancia asignada a FP o FN es la misma.

4.6 Promedios: micro, macro y ponderado

Dado que las métricas de recall, precisión y F1 pueden calcularse de a una por clase, es común encontrar promedios para obtener un recall promedio, una precisión promedio y un F1 promedio. En esta sección se enumeran las tres estrategias para promediar más comunes.

4.6.1 Micro promedio

Calcula las métricas globales sumando los verdaderos positivos, falsos positivos y falsos negativos de todas las clases:

$$\text{Precision}_{\text{micro}} = \frac{\sum_i TP_i}{\sum_i (TP_i + FP_i)} \quad \text{Recall}_{\text{micro}} = \frac{\sum_i TP_i}{\sum_i (TP_i + FN_i)}$$

El F1-score micro también se calcula con estos valores agregados:

$$F1_{\text{micro}} = \frac{2 \cdot \text{Precision}_{\text{micro}} \cdot \text{Recall}_{\text{micro}}}{\text{Precision}_{\text{micro}} + \text{Recall}_{\text{micro}}}$$

Este promedio da más peso a las clases con más muestras, por lo que puede estar sesgado hacia clases mayoritarias.

4.6.2 Macro promedio

Calcula la métrica por clase y luego promedia:

$$\text{Precision}_{\text{macro}} = \frac{1}{N} \sum_i \text{Precision}_i \quad \text{Recall}_{\text{macro}} = \frac{1}{N} \sum_i \text{Recall}_i \quad F1_{\text{macro}} = \frac{1}{N} \sum_i F1_i$$

Aquí, todas las clases tienen el mismo peso, sin importar cuántas muestras tengan. Es útil para evaluar el desempeño en clases minoritarias.

4.6.3 Promedio ponderado

También calcula la métrica por clase, pero pondera cada una por su cantidad de muestras n_i :

$$\text{Precision}_{\text{weighted}} = \sum_i w_i \cdot \text{Precision}_i \quad \text{donde } w_i = \frac{n_i}{\sum_j n_j}$$

Lo mismo aplica para Recall y F1. Este promedio intenta balancear el sesgo hacia clases grandes, sin descuidar las pequeñas.

4.7 Métricas personalizadas

Las métricas clásicas como precisión, recall o F1-score surgen de combinaciones particulares de los valores del conjunto de confusión: TP, FP, FN y TN. Sin embargo, es posible construir métricas personalizadas que se adapten mejor a los objetivos específicos de una tarea.

Por ejemplo, en ciertos contextos puede ser más costoso cometer un falso negativo que un falso positivo (como en un test médico), o puede interesar una métrica que penalice fuertemente los errores. Algunos ejemplos:

- Penalización a FP y FN :

$$\text{Métrica}_{\text{ejemplo}} = \frac{TP - \alpha \cdot FP - \beta \cdot FN}{TP + FP + FN}$$

Donde α y β son hiperparámetros que penalizan en distinta medida los FP y FN. Esta métrica puede adaptarse a problemas con distintos costos asociados a los errores.

- Precisión extendida con ponderación sobre TN:

$$\text{Precisión}_{\text{ext}} = \frac{TP + \gamma \cdot TN}{TP + FP + \gamma \cdot TN}$$

Puede ser útil si se desea premiar la correcta clasificación de negativos.

- De manera más general:

$$\text{Métrica}_{\text{custom}} = f(TP, FP, FN, TN)$$

Donde f es una función arbitraria que puede incluir sumas, productos, razones, logaritmos, pesos o condiciones lógicas, dependiendo de las prioridades del problema.

Diseñar una métrica personalizada es una forma de alinear la optimización del modelo con los objetivos reales del negocio o del sistema. Las métricas estándar pueden no ser suficientes en situaciones donde las clases están altamente desbalanceadas, los costos de los errores no son simétricos o el modelo será usado en un contexto con requerimientos regulatorios o éticos específicos.

4.8 Curva ROC y AUC

La curva ROC (Receiver Operating Characteristic) representa la capacidad de un modelo para distinguir entre clases a medida que se varía uno de los hiperparámetros clave en cualquier problema de clasificación: el umbral de decisión.

En esta curva, se grafica sobre el eje X el False Positive Rate (FPR), que es la proporción de negativos reales clasificados incorrectamente como positivos y sobre el eje Y el True Positive Rate (TPR), que es la proporción de positivos reales clasificados correctamente.

$$\text{TPR} = \frac{TP}{TP + FN}, \quad \text{FPR} = \frac{FP}{FP + TN}$$

A medida que varía el umbral, varía la salida del modelo (a menor umbral, mayor cantidad de predicciones positivas, y a mayor umbral, mayor cantidad de predicciones negativas) y por tanto FP , FN , TP y TN . Consecuentemente, también varían TPR y FPR . Para construir una curva lo suficientemente suave es necesario variar el umbral entre 0 y 1 con una cantidad de pasos adecuada.

Para un umbral igual a 0, el modelo siempre predice la clase positiva. Por lo que $FN = 0$ y resulta $TPR = 1$. Además $TN = 0$ así que $FPR = 1$ también. Para un umbral igual a 1, el modelo siempre predice la clase negativa. Por lo que $TP = 0$ entonces $TPR = 0$ y $FP = 0$ por lo que $FPR = 0$ también. Los puntos $(0, 0)$ y $(1, 1)$ son parte de la gráfica siempre.

Si bien el modelo de probabilidad de pertenencia a cada clase es el mismo para todos los casos, variar el umbral de decisión es cambiar también el clasificador. Un clasificador está representado sólo como un punto en la curva. El clasificador perfecto es aquel que cae en el punto $(0, 1)$ de la curva, lo que significa que existe un umbral para el cual el modelo de probabilidad presenta un desempeño máximo, con una muy alta TPR y baja FPR . En la mayoría de los casos, este punto no pertenecerá a la curva ROC, pero sí se puede encontrar aquel clasificador que esté más cerca de trabajar en ese punto de operación. Un clasificador totalmente erróneo es aquel que cae en el punto $(1, 0)$, ya que solo produce falsos. Y un clasificador que cae sobre la diagonal que va desde $(0, 0)$ a $(1, 1)$ presenta el mismo desempeño que un modelo de predicción aleatoria, puesto que produce una tasa de tantos falsos como verdaderos por igual. En ese caso, gran parte del esfuerzo de aprendizaje estaría desperdiciado, ya que una moneda lanzada al azar obtendría resultados similares [11].

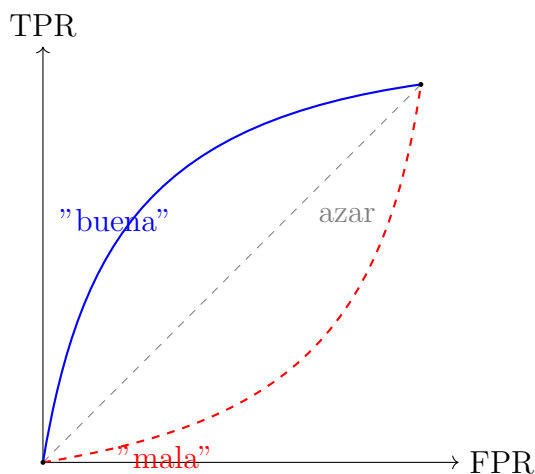


Figura 20: Ejemplo de curva ROC.

4.8.1 AUC - Área bajo la curva

El AUC es el área bajo la curva ROC, por lo que puede cuantificar el desempeño del modelo de umbral óptimo, veamos:

- $AUC = 1$: clasificación perfecta, ya que significa que el punto $(0, 1)$ pertenece a la curva ROC.
- $AUC = 0.5$: desempeño equivalente al azar, ya que la curva será muy similar a la punteada de la figura 20.
- $AUC < 0.5$: el modelo predice peor que el modelo aleatorio.

Un elevado AUC debe ser acompañado de la optimización del umbral.

5 Balanceo de clases

En muchos conjuntos de datos reales, especialmente en tareas de clasificación binaria, puede suceder que una clase esté significativamente más representada que la otra. Esta situación se conoce como desbalanceo de clases y puede llevar a que los modelos de aprendizaje presenten un sesgo hacia la clase mayoritaria [9].

Por ejemplo, si el 95% de los ejemplos pertenecen a la clase 0 y solo el 5% a la clase 1, un modelo podría obtener una alta exactitud simplemente prediciendo siempre la clase 0, sin aprender nada útil sobre la clase minoritaria. Por esta razón, es importante considerar estrategias de *balanceo de clases* para el conjunto de datos de entrenamiento, de forma de poder generalizar correctamente hacia los datos de prueba.

5.1 Técnicas para balancear clases

Submuestreo de la clase mayoritaria (undersampling): consiste en reducir el número de ejemplos de la clase mayoritaria, seleccionando una muestra representativa de ella. Como desventaja, puede llevar a la pérdida de información relevante.

Sobremuestreo de la clase minoritaria (oversampling): implica replicar o sintetizar ejemplos de la clase minoritaria para igualar la proporción con la clase mayoritaria. Técnicas comunes son Random Oversampling y SMOTE (Synthetic Minority Over-sampling Technique) [5].

Ponderación de clases (class weighting): se puede asignar un peso mayor a los errores cometidos sobre la clase minoritaria durante el entrenamiento del modelo. Muchos algoritmos, como `LogisticRegression` de `scikit-learn`, permiten este ajuste mediante el parámetro `class_weight` [14].

Generación de datos sintéticos: además de SMOTE, existen otras variantes como ADASYN, Borderline-SMOTE, etc., que generan nuevos ejemplos sintéticos para mejorar la representación de la clase minoritaria.

Ajuste de umbral de decisión: si el modelo proporciona probabilidades, puede modificarse el umbral de clasificación para favorecer la detección de la clase minoritaria (por ejemplo, prediciendo clase 1 si $p > 0.3$ en lugar de $p > 0.5$).

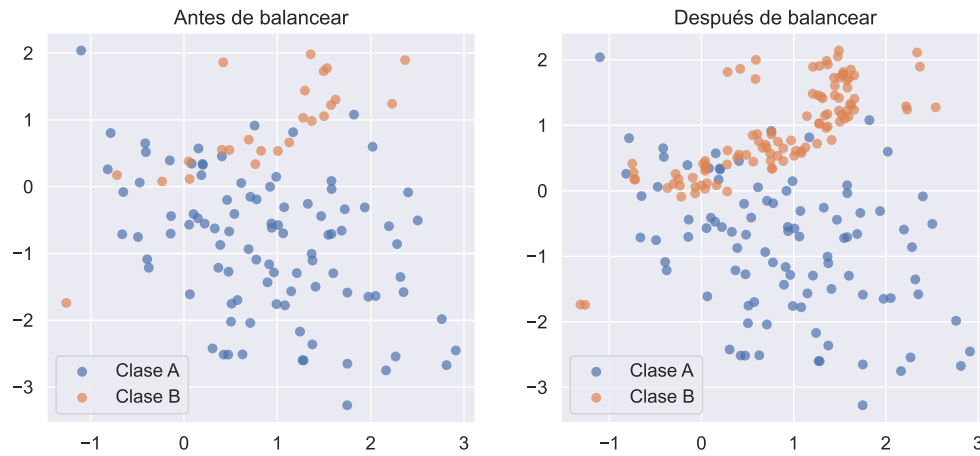


Figura 21: Conjunto de entrenamiento antes y después de balancear por oversampling de clase B.

5.2 Métricas adecuadas

Como se enunciaba en la sección 5, accuracy no es una métrica adecuada para un problema de clases desbalanceadas. Es importante elegir de la sección 4, y acorde al contexto del problema, una métrica que sí sea apta para este tipo de problemas, como lo podrían ser el recall, la precisión, el F1 Score o alguna métrica personalizada, entre otras.

6 Explicabilidad de modelos

La explicabilidad se refiere a la capacidad de describir, entender y justificar las decisiones que toma un modelo. Modelos como regresión lineal o árboles de decisión son inherentemente explicables (llamados modelos *white-box*), mientras que redes neuronales profundas o ensambles suelen considerarse *black-box*, debido a su elevada complejidad.

La explicabilidad también está relacionada con la equidad y la transparencia. Un modelo que no puede ser interpretado es difícil de auditar y puede perpetuar sesgos. Por esto, en ciertos dominios, existen regulaciones que exigen la posibilidad de justificar automáticamente las decisiones tomadas por sistemas de IA [12].

Algunos métodos de explicabilidad son:

- **Importancia de características:** mide el impacto de cada variable en la predicción del modelo. En árboles, por ejemplo, se usa la reducción de impureza.
- **SHAP (SHapley Additive exPlanations):** basado en teoría de juegos, asigna a cada variable una contribución a la predicción.
- **LIME (Local Interpretable Model-agnostic Explanations):** aproxima el modelo de forma local con uno interpretable para explicar predicciones individuales.
- **Partial Dependence Plots (PDP):** muestran cómo varía la predicción esperada al cambiar una característica, manteniendo las demás fijas.
- **Contrafactuales:** muestran cómo debería cambiar una entrada para alterar una decisión del modelo.

En la práctica, la explicabilidad se analizará con SHAP [17] [12].

7 Comparación de modelos y ajuste fino

7.1 Baseline models

Un modelo baseline (o de referencia) es un punto de partida para evaluar modelos más sofisticados. Su propósito no es lograr un rendimiento óptimo, sino establecer un estándar mínimo que cualquier modelo debe superar para justificar su complejidad.

7.1.1 Tipos de modelos baseline

Modelos estadísticos simples Son reglas básicas que no requieren aprendizaje automático. Ejemplos comunes incluyen:

- Predecir siempre la clase mayoritaria.
- Tomar el promedio histórico de la variable de salida.
- Utilizar un modelo lineal con una cantidad muy acotada de variables predictoras.

Usualmente son demasiado ingenuos y extremadamente poco representativos.

Procesos actuales no automatizados Muchas veces el modelo baseline se construye

a partir del trabajo manual, heurístico o experto que se realiza actualmente. Por ejemplo:

- Un sistema de diagnóstico médico puede tener como baseline la precisión de un médico humano.
- En un sistema de detección de fraudes, el baseline puede ser el conjunto de reglas utilizadas por un equipo de analistas.
- En problemas con experiencia de usuario implícita, el juicio humano puede ser una línea base válida aunque no esté formalizado en código.

7.1.2 Comparación con modelos complejos

Un modelo adecuado debe superar al baseline en términos de métrica principal (accuracy, F1, MSE, etc.). costo computacional (cuando sea relevante) y generalización a nuevos datos. Superar al baseline no solo justifica la automatización, sino que también ofrece evidencia de mejora real respecto de los procesos actuales.

7.2 Criterios para comparación de modelos

A la hora de comparar modelos, se debe mantener una metodología coherente y criterios claros. Algunos puntos clave a considerar son:

- **Dataset:** asegurarse de que todos los modelos sean entrenados y evaluados sobre los mismos conjuntos de datos (entrenamiento, validación y test). Esto garantiza una comparación justa.

- **Métrica:** elegir **una** métrica adecuada que represente correctamente el objetivo del problema (por ejemplo, F1 de clase 0, accuracy, etc.). Esta métrica debe ser optimizada. Aquel modelo con mejor métrica es el de mejor desempeño. Observar otras métricas no es un error, pero no deben tenerse en cuenta para la decisión.
- **Complejidad:** considerar la complejidad del modelo (por ejemplo, número de parámetros, profundidad de árboles, regularización), lo cual puede afectar tanto la generalización como el mantenimiento.
- **Interpretabilidad:** algunos modelos (como árboles de decisión o regresión lineal) permiten interpretar fácilmente cómo se toman las decisiones, lo cual puede ser relevante en algunos contextos (como salud o finanzas). Ante igualdad de métricas, siempre es preferible un modelo más interpretable.
- **Costo computacional:** evaluar el costo en tiempo de entrenamiento, predicción y recursos computacionales necesarios (si se requiere realizar predicciones en 5 milisegundos, ¿este modelo es indicado?; ¿se cuenta con el hardware adecuado?) Este factor puede ser determinante en aplicaciones en tiempo real o de gran escala.

8 Ajuste fino

El ajuste fino (*fine-tuning*) es el proceso mediante el cual se optimizan los hiperparámetros o se ajustan los parámetros de un modelo con el objetivo de mejorar su rendimiento en una tarea o conjunto de datos específicos. Este proceso puede darse en dos contextos principales:

- **Optimización de hiperparámetros:** consiste en encontrar la mejor combinación de hiperparámetros del modelo (como tasa de aprendizaje, profundidad, regularización, etc.).
- **Ajuste de modelos preentrenados:** especialmente en modelos de redes neuronales profundas, implica tomar un modelo general y especializarlo en una tarea específica.

El objetivo principal es mejorar la métrica obtenida, seguido de encontrar modelos más robustos y consistentes.

8.1 Optimización de hiperparámetros

Para optimizar hiperparámetros es necesario contar con cinco elementos:

- un estimador de $\hat{y}$.
- una métrica a mejorar.
- un esquema de validación cruzada.
- un espacio de valores para los hiperparámetros.
- un método de búsqueda para muestrear el espacio de valores.

8.2 Métodos de búsqueda

Los métodos de búsqueda más comunes son: grid search, random search y optimización bayesiana.

Búsqueda en grilla (grid search)

La búsqueda en grilla consiste en definir un conjunto de valores posibles para cada hiperparámetro y luego evaluar el rendimiento del modelo en todas las combinaciones posibles. Es un método sistemático y exhaustivo, especialmente útil cuando se trabaja con pocos hiperparámetros y se desea analizar detalladamente su efecto sobre el desempeño del modelo.

Uno de los principales beneficios de este enfoque es su simplicidad: es fácil de implementar y de interpretar, y asegura que se exploran todas las combinaciones dentro del espacio definido. Sin embargo, esta exhaustividad es también su mayor limitación, ya que el número de combinaciones crece exponencialmente con el número de hiperparámetros,

haciendo que el método sea computacionalmente costoso y muchas veces ineficiente. A menudo se termina evaluando configuraciones que no aportan mejoras sustanciales, especialmente cuando ciertos hiperparámetros tienen más influencia que otros. A pesar de estas limitaciones, la búsqueda en grilla sigue siendo popular en escenarios donde se dispone de recursos computacionales suficientes y se necesita un análisis exhaustivo.

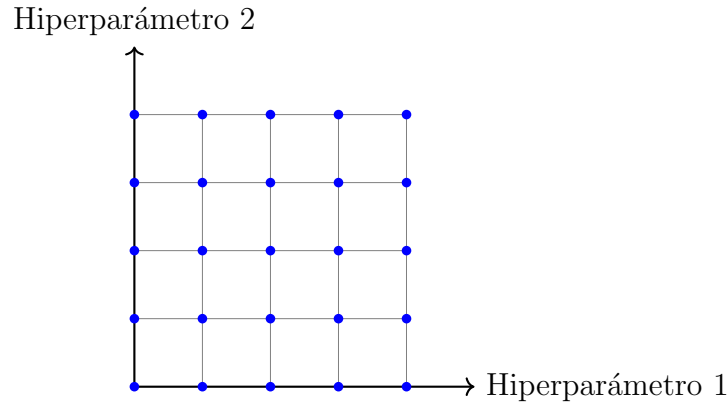


Figura 22: Exploración exhaustiva del espacio en una grilla regular.

Búsqueda aleatoria (random search)

A diferencia del método anterior, la búsqueda aleatoria no evalúa todas las combinaciones posibles, sino que selecciona al azar un número fijo de combinaciones a partir de distribuciones definidas para cada hiperparámetro.

Esta estrategia permite una exploración más amplia del espacio de búsqueda en menos tiempo, lo que la hace particularmente eficaz cuando algunos hiperparámetros tienen una influencia mucho mayor que otros. En lugar de desperdiciar evaluaciones en regiones poco prometedoras, la búsqueda aleatoria permite cubrir más terreno y tiene mayor probabilidad de encontrar configuraciones cercanas al óptimo en menos iteraciones que la búsqueda en grilla [3]. Aunque no garantiza encontrar la mejor combinación posible y sus resultados pueden variar entre ejecuciones, es una alternativa eficiente cuando se cuenta con un presupuesto computacional limitado o se enfrentan problemas con un espacio de búsqueda amplio. Además, es posible detener el proceso en cualquier momento y quedarse con la mejor configuración encontrada hasta ese punto, lo cual le otorga una flexibilidad adicional.

Optimización bayesiana

La optimización bayesiana representa un enfoque más avanzado y adaptativo para la búsqueda de hiperparámetros. En lugar de seleccionar combinaciones al azar o de forma exhaustiva, este método construye un modelo probabilístico del rendimiento del modelo (comúnmente un proceso de Gauss) y lo utiliza para predecir qué configuraciones de hiperparámetros podrían producir mejores resultados. Se equilibra la exploración de regiones poco conocidas del espacio con la explotación de regiones que ya han mostrado buenos desempeños. Esta metodología es altamente eficiente en términos de número de evaluaciones

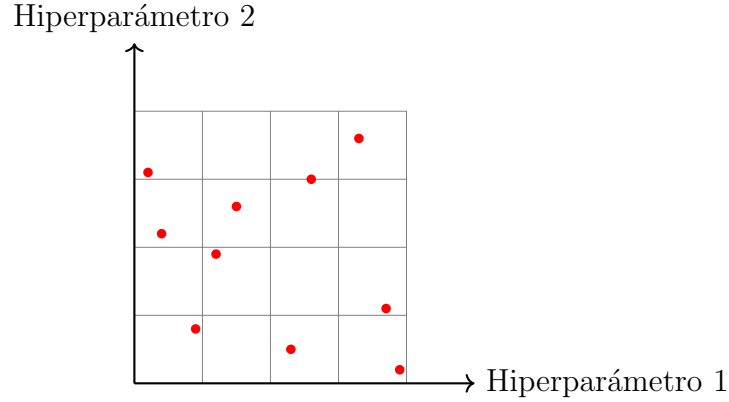


Figura 23: Exploración aleatoria del espacio de hiperparámetros.

necesarias, y puede identificar buenas configuraciones con relativamente pocas pruebas, lo cual la convierte en una opción ideal cuando entrenar el modelo es costoso o lento. No obstante, su principal desventaja radica en la complejidad: requiere mayor esfuerzo computacional por iteración y un conocimiento más técnico para su implementación. Además, supone que la función de evaluación es suave y continua, una suposición que puede no cumplirse en todos los casos. A pesar de esto, la optimización bayesiana ha demostrado ser especialmente efectiva en problemas de optimización global y se encuentra implementada en bibliotecas modernas como `Hyperopt`, `Optuna` y `scikit-optimize`, facilitando su adopción en la práctica [16].

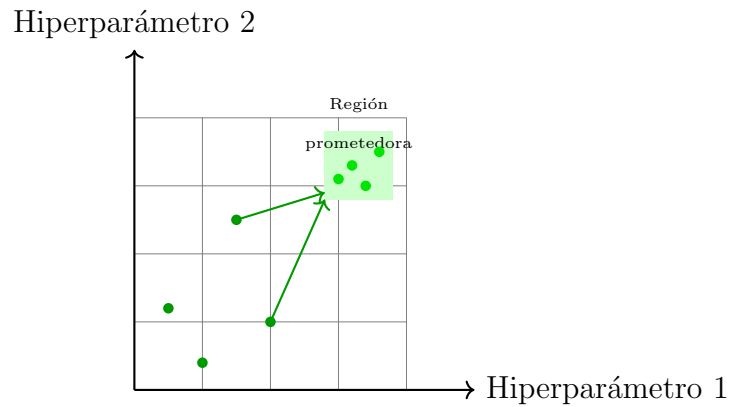


Figura 24: Exploración inicial seguida por concentración en una región prometedora del espacio de búsqueda.

9 Introducción a redes neuronales

Las redes neuronales artificiales son una extensión de los modelos clásicos de aprendizaje automático. Su diseño está inspirado en el funcionamiento del cerebro humano, pero su fundamento es puramente matemático. Esta sección introduce sus componentes básicos, arquitectura, funcionamiento y entrenamiento. Se intenta brindar un enfoque tanto conceptual como matemático.

9.1 Del aprendizaje automático al aprendizaje profundo

En el aprendizaje automático tradicional, los modelos como regresión logística, máquinas de soporte vectorial o árboles de decisión son eficaces cuando los datos pueden representarse adecuadamente mediante características diseñadas manualmente. Sin embargo, en problemas de alta dimensionalidad o en tareas como procesamiento de imágenes, texto o audio, la extracción manual de características se vuelve limitada. Surge entonces el aprendizaje profundo: una familia de métodos basada en redes neuronales profundas que pueden aprender representaciones jerárquicas directamente a partir de los datos, y convirtiéndose por lo tanto en métodos más *black-box*, que requieren más atención a la hora de interpretarse (en los casos en los que se pueda).

La principal diferencia entre los modelos clásicos y las redes neuronales profundas radica en la automatización del aprendizaje de características (feature learning), que en deep learning ocurre de forma conjunta con el aprendizaje del modelo, a través de múltiples capas de representación.

9.2 Los modelos lineales no son suficientes

El ejemplo del aprendizaje de la función XOR ilustra claramente por qué un modelo lineal (como una regresión logística) no puede resolver este problema: la función XOR no es linealmente separable. En particular, al entrenar tal modelo con el conjunto de entrenamiento $\{[0, 0], [0, 1], [1, 0], [1, 1]\}$ y usar un criterio de error cuadrático medio, la solución óptima encontrada (resolviendo las ecuaciones normales) es $w = 0$ y $b = 0.5$, lo que implica que el modelo predice $f(x) = 0.5$ para cualquier entrada. Esto demuestra que un modelo lineal no logra capturar la estructura de XOR [7].

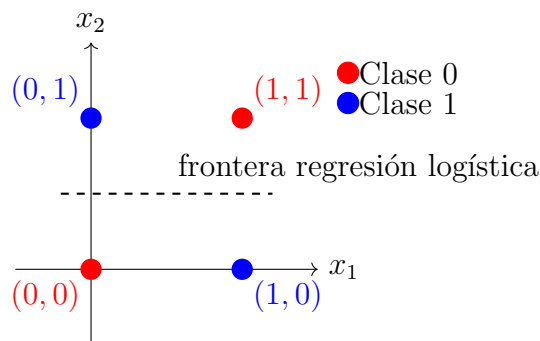


Figura 25: Dataset XOR.

En la práctica se retoma este problema con aprendizaje profundo, después de primero discutir la arquitectura, funciones de pérdida y optimizadores de una red neuronal.

9.3 La neurona artificial

La unidad básica de una red neuronal es la **neurona artificial**, también conocida como *perceptrón*. Matemáticamente, una neurona toma un vector de entrada $x \in R^n$, lo combina linealmente con un vector de pesos $w \in R^n$, suma un sesgo (bias) $b \in R$, y aplica una función de activación f no lineal para generar la salida:

$$z = w^\top x + b, \quad a = f(z)$$

Donde a es la activación de la neurona.

9.4 Conexión entre neuronas y capas

Una red neuronal se compone de muchas neuronas conectadas entre sí. Estas conexiones forman estructuras llamadas **capas**. Cada capa recibe como entrada el vector de activaciones de la capa anterior y produce un nuevo vector de activaciones.

- La **capa de entrada** recibe los datos originales.
- Las **capas ocultas** procesan internamente la información, transformando representaciones a través de funciones no lineales.
- La **capa de salida** produce el resultado final del modelo, como una probabilidad en clasificación o un valor continuo en regresión.

En una red **fully connected**, cada neurona de una capa está conectada con todas las neuronas de la siguiente. Si una capa tiene n neuronas de entrada y m neuronas de salida, entonces hay $n \times m$ pesos y m sesgos.

9.5 Entrenamiento de una red neuronal

Entrenar una red consiste en ajustar los pesos y biases para que la salida del modelo se aproxime lo más posible a la salida deseada. La gran diferencia con los modelos lineales es que la introducción de funciones no lineales genera que las funciones de pérdida más habituales se vuelvan **no convexas**. Esto significa que deben entrenarse utilizando *optimizadores iterativos basados en gradiente*, en lugar de los **solvers de ecuaciones lineales**.

La optimización convexa converge desde cualquier conjunto inicial de parámetros (en teoría; en la práctica es robusta, pero puede presentar problemas numéricos). En cambio, el *descenso estocástico por gradiente* aplicado a funciones de pérdida no convexas **no posee garantías de convergencia** y es sensible a los valores iniciales de los parámetros. En las redes neuronales *feed-forward*, es importante inicializar todos los **pesos** con valores pequeños y aleatorios. Los **sesgos** pueden inicializarse en cero o con valores pequeños

positivos. Por ahora, basta con entender que el algoritmo de entrenamiento casi siempre se basa en usar el **gradiente** para descender en la función de costo de una forma u otra.

Nacen entonces dos sistemas de ecuaciones necesarios para inferir y calcular gradientes: el sistema de *propagación hacia adelante* (*feedforward*) que, a partir de una entrada proporciona la salida $\hat{y}$ y las ecuaciones *propagación hacia atrás* (*backpropagation*) que calculan los gradientes como vectores cuyas componentes son las derivadas de la función de pérdida $\mathcal{L}(\hat{y}, y)$ respecto de cada uno de los coeficientes del modelo. Se tratan en las secciones 9.9 y 9.10.

Las funciones de pérdida más habituales se discuten en la sección 9.6, mientras que los optimizadores basados en gradiente (y usualmente, *SGD*) se comentan en la sección 9.7.

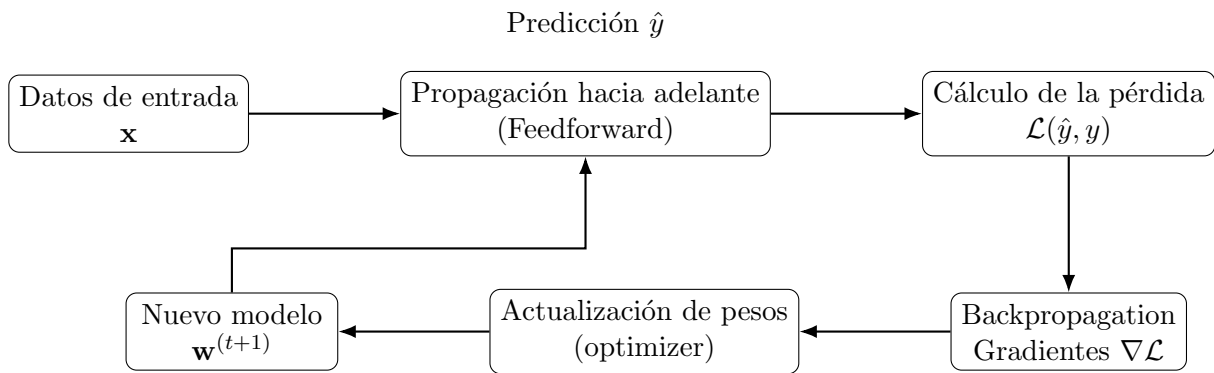


Figura 26: Diagrama del proceso de entrenamiento de una red neuronal.

9.6 Funciones de pérdida

La función de pérdida, así como en todos los modelos que venimos trabajando, es necesaria para el cálculo de gradientes en la optimización de los coeficientes de la red. Según el tipo de tarea, existen diferentes funciones de pérdida apropiadas¹. Las más utilizadas suelen ser el MSE en regresión y la entropía cruzada en clasificación.

9.7 Optimizadores

Los optimizadores actualizan los pesos del modelo para minimizar la función de pérdida. Algunos optimizadores comunes en TensorFlow² son:

- **SGD (Stochastic Gradient Descent)**: simple pero efectivo, es el método desarrollado en 2.4.4

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}$$

- **Momentum SGD**: acelera SGD en direcciones relevantes:

$$v_t = \gamma v_{t-1} + \eta \nabla_{\theta} \mathcal{L}, \quad \theta \leftarrow \theta - v_t$$

donde:

¹https://www.tensorflow.org/api_docs/python/tf/keras/losses

²https://www.tensorflow.org/api_docs/python/tf/keras/optimizers

- v_t : velocidad (momento) acumulado
- γ : factor de momentum (0 a 1)
- η : tasa de aprendizaje
- **Adagrad**: adapta la tasa de aprendizaje de cada parámetro según la historia de los gradientes:

$$G_t = G_{t-1} + (\nabla_{\theta} \mathcal{L})^2, \quad \theta \leftarrow \theta - \frac{\eta}{\sqrt{G_t} + \epsilon} \nabla_{\theta} \mathcal{L}$$

donde:

- G_t : suma acumulada de los gradientes al cuadrado
- ϵ : valor pequeño para evitar división por cero
- **RMSProp**: similar a Adagrad, pero usando una media móvil exponencial de los gradientes al cuadrado, lo que estabiliza el aprendizaje en problemas ruidosos.
- **Adam (Adaptive Moment Estimation)**: combina Momentum y RMSProp:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \nabla_{\theta} \mathcal{L}, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) (\nabla_{\theta} \mathcal{L})^2$$

donde:

- m_t : estimación del primer momento (media) del gradiente
- v_t : estimación del segundo momento (varianza) del gradiente
- β_1, β_2 : factores de decaimiento para las medias móviles

entre otros.

9.8 Arquitecturas de modelos

Existen diversas arquitecturas de modelos en **Keras** para distintos problemas³:

- **Red neuronal densa (Dense)**: todas las neuronas de una capa se conectan a todas las de la siguiente.
- **Redes convolucionales (CNN)**: extracción de características espaciales, común en imágenes.
- **Redes recurrentes (RNN)**: modelado de secuencias temporales (texto, series de tiempo).
- **LSTM (Long Short-Term Memory)**: una arquitectura de red neuronal recurrente (RNN) diseñada para aprender dependencias a largo plazo. Incorpora puertas (de entrada, olvido y salida) que regulan el flujo de información, lo que permite mantener información relevante durante muchas secuencias de tiempo y evita el problema del desvanecimiento del gradiente.

entre otros.

³https://www.tensorflow.org/guide/keras/sequential_model

9.9 Ecuaciones feed-forward

En una red neuronal multicapa, el proceso de propagación hacia adelante (*feed-forward*) consiste en calcular las activaciones de cada capa a partir de las activaciones de la capa anterior.

Sea una red con:

- L capas numeradas de 1 (entrada) a L (salida).
- n_l neuronas en la capa l .
- Función de activación $g_l(\cdot)$ para la capa l (excepto la de entrada, que simplemente pasa los datos).

El cálculo para cada capa $l = 2, \dots, L$ es:

$$z_j^{(l)} = \sum_{i=1}^{n_{l-1}} w_{ji}^{(l)} a_i^{(l-1)} + b_j^{(l)}, \quad j = 1, \dots, n_l, \quad (\text{pre-activación})$$
$$a_j^{(l)} = g_l(z_j^{(l)}), \quad j = 1, \dots, n_l, \quad (\text{activación})$$

donde:

- $a_i^{(l-1)}$ es la activación de la neurona i en la capa anterior.
- $w_{ji}^{(l)}$ es el peso que conecta la neurona i de la capa $l - 1$ con la neurona j de la capa l .
- $b_j^{(l)}$ es el sesgo (*bias*) de la neurona j en la capa l .
- $g_l(\cdot)$ es la función de activación de la capa l .

En forma matricial, para la capa l :

$$\mathbf{z}^{(l)} = W^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \quad \mathbf{a}^{(l)} = g_l(\mathbf{z}^{(l)}),$$

donde $W^{(l)} \in R^{n_l \times n_{l-1}}$, $\mathbf{b}^{(l)} \in R^{n_l}$ y g_l actúa componente a componente.

9.10 Ecuaciones back-propagation

El algoritmo de *back-propagation* permite calcular el gradiente del error respecto de cada peso y sesgo de la red neuronal, propagando la información de manera inversa: desde la salida hacia la entrada. Por lo tanto es un algoritmo que primero aplica a la capa de salida y luego va aplicando sucesivamente a cada capa anterior, con el uso exhaustivo de la regla de la cadena para obtener derivadas.

Sea:

- L : número total de capas de la red (incluyendo entrada y salida).
- $a_j^{(l)}$: activación de la neurona j en la capa l .

Capa de entrada Capa oculta Capa de salida

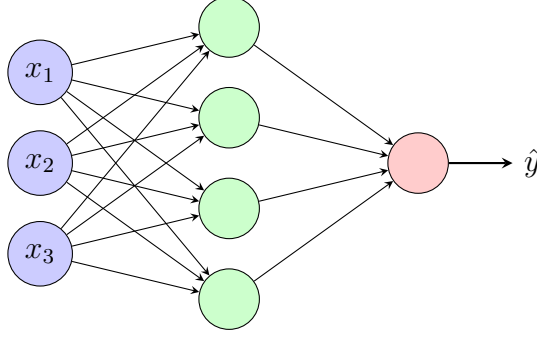


Figura 27: Red neuronal feed-forward con una sola salida $\hat{y}$.

- $z_j^{(l)}$: valor de pre-activación correspondiente.
- $w_{ji}^{(l)}$: peso que conecta la neurona i de la capa $l - 1$ con la neurona j de la capa l .
- $b_j^{(l)}$: sesgo de la neurona j en la capa l .
- $g_l(\cdot)$: función de activación de la capa l .
- C : función de pérdida.

Error en la capa de salida Para cada neurona j de la capa de salida L :

$$\delta_j^{(L)} = \frac{\partial C}{\partial a_j^{(L)}} \cdot g_L'(z_j^{(L)}),$$

donde $g_L'(\cdot)$ es la derivada de la función de activación.

Propagación hacia capas anteriores Para $l = L - 1, L - 2, \dots, 2$:

$$\delta_j^{(l)} = \left(\sum_{k=1}^{n_{l+1}} w_{kj}^{(l+1)} \delta_k^{(l+1)} \right) \cdot g_l'(z_j^{(l)}).$$

Gradientes respecto a los pesos y sesgos

$$\begin{aligned} \frac{\partial C}{\partial w_{ji}^{(l)}} &= a_i^{(l-1)} \delta_j^{(l)}, \\ \frac{\partial C}{\partial b_j^{(l)}} &= \delta_j^{(l)}. \end{aligned}$$

Forma matricial En notación matricial:

$$\begin{aligned}\boldsymbol{\delta}^{(L)} &= \nabla_{\mathbf{a}^{(L)}} C \odot g'_L(\mathbf{z}^{(L)}), \\ \boldsymbol{\delta}^{(l)} &= \left(W^{(l+1)T} \boldsymbol{\delta}^{(l+1)} \right) \odot g'_l(\mathbf{z}^{(l)}), \\ \frac{\partial C}{\partial W^{(l)}} &= \boldsymbol{\delta}^{(l)} \mathbf{a}^{(l-1)T}, \\ \frac{\partial C}{\partial \mathbf{b}^{(l)}} &= \boldsymbol{\delta}^{(l)},\end{aligned}$$

donde $\odot$ denota el producto elemento a elemento.

Una vez obtenidas todas las derivadas parciales que componen los vectores gradiente, se puede utilizar un optimizador (9.7) para actualizar los coeficientes de forma iterativa.

Capa de entrada Capa oculta Capa de salida

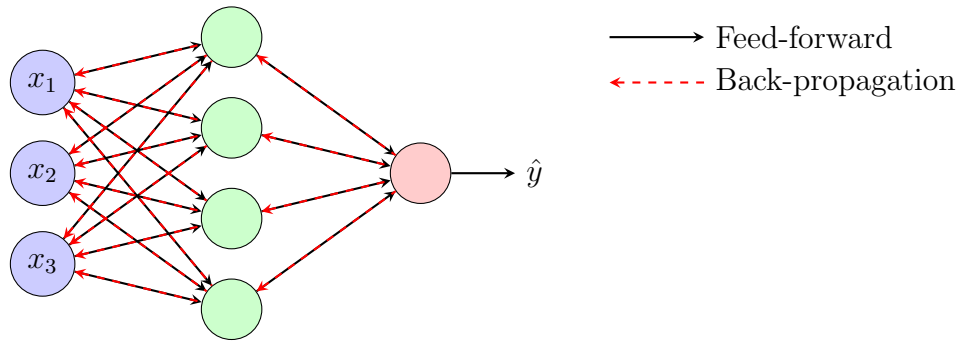


Figura 28: Flujo de cálculo en una red neuronal: propagación hacia adelante (negro) y retropropagación del error (rojo).

9.11 Funciones de activación

Las funciones de activación más comunes en redes neuronales son:

- Función escalón (step function)
- Función sigmoide (logística)
- Tangente hiperbólica (tanh)
- ReLU (Rectified Linear Unit)
- Leaky ReLU
- Softmax

9.11.1 Función escalón

La función escalón es una función no diferenciable definida por:

$$g(x) = \begin{cases} 1 & \text{si } x \geq 0 \\ 0 & \text{si } x < 0 \end{cases}$$

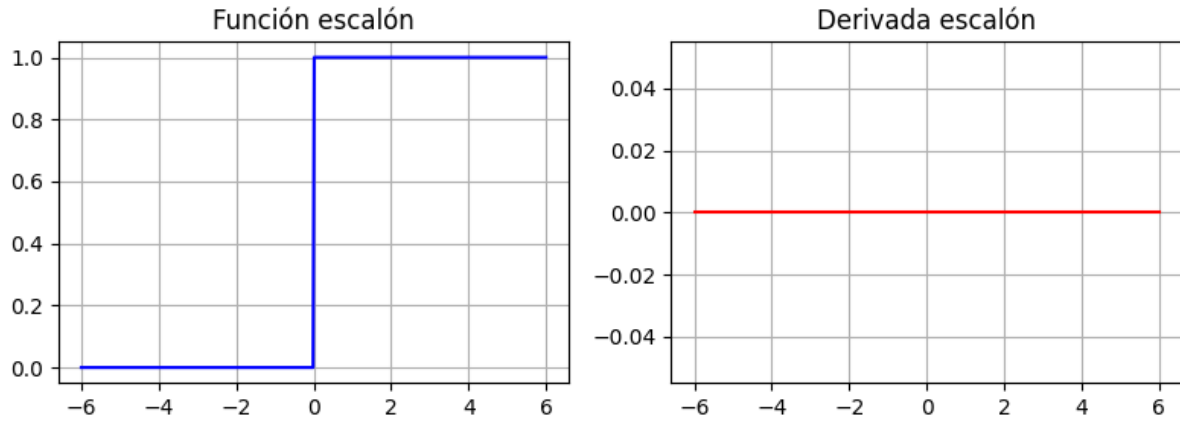


Figura 29: Función escalón y su derivada

Como la derivada es cero casi en todos lados, esta función no permite el aprendizaje mediante descenso por gradiente.

9.11.2 Función sigmoide

La función sigmoide está dada por:

$$g(x) = \frac{1}{1 + e^{-x}}$$

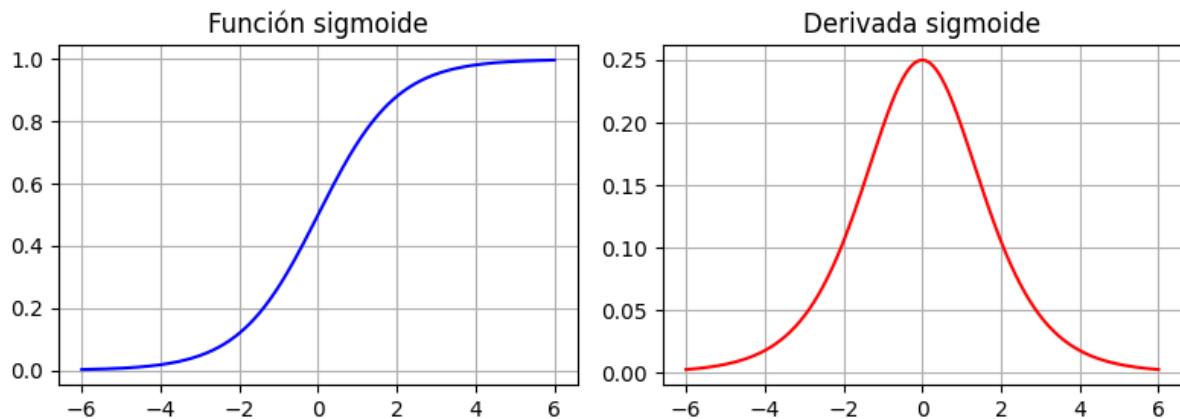


Figura 30: Función sigmoide y su derivada

Aunque es diferenciable y suave, la sigmoide puede presentar el problema de *gradientes saturados*, lo que dificulta el entrenamiento de redes profundas.

Ya que su salida es un rango continuo entre 0 y 1, es la función de activación por excelencia para las capas de salida en problemas de clasificación (al igual que en la regresión logística). La función softmax es su generalización hacia problemas multiclase.

9.11.3 Tangente hiperbólica (tanh)

La tangente hiperbólica está definida como:

$$g(x) = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

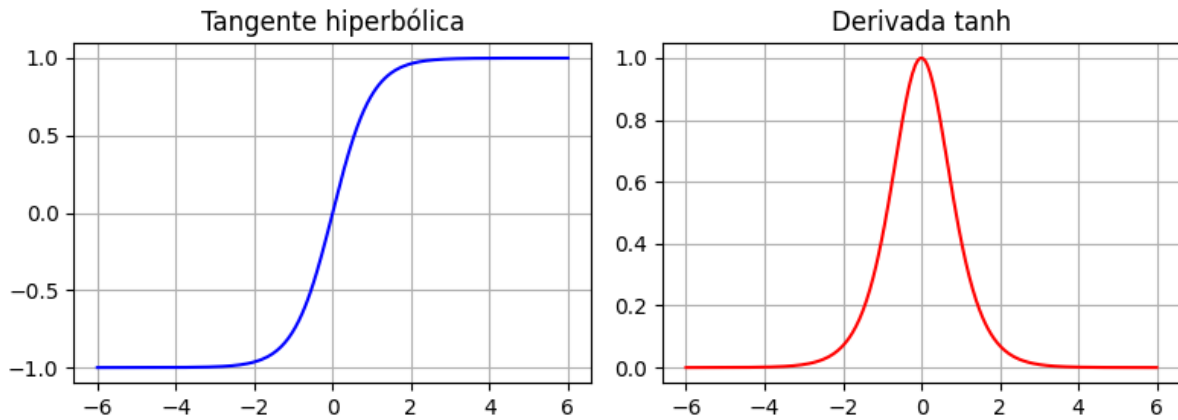


Figura 31: Función tangente hiperbólica y su derivada

La función tanh tiene salida centrada en cero y gradientes mayores que la sigmoide, pero también puede saturar para valores extremos.

9.11.4 ReLU (Rectified Linear Unit)

La función ReLU está definida como:

$$g(x) = \max(0, x)$$

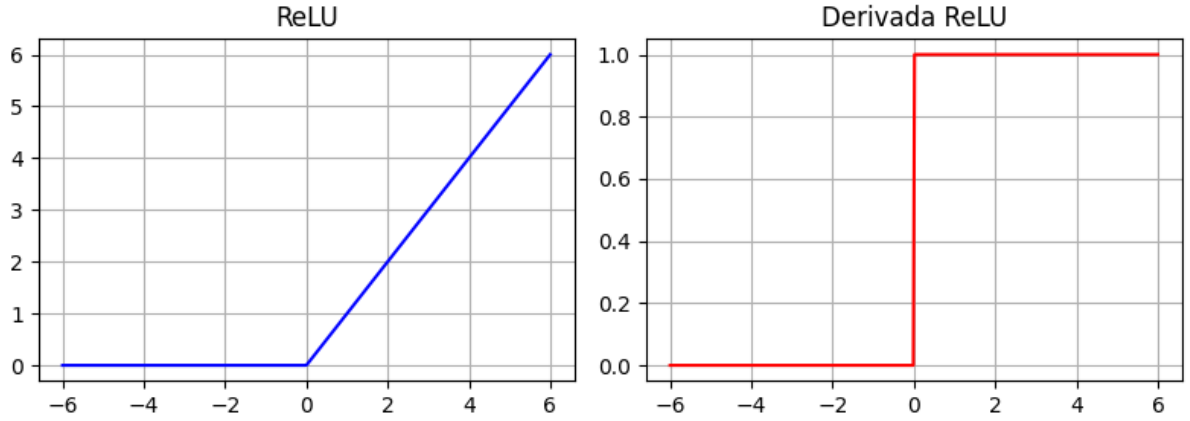


Figura 32: Función ReLU y su derivada

ReLU es computacionalmente eficiente y ayuda a evitar el problema de gradientes saturados, aunque puede presentar el problema de neuronas muertas, ya que ante entradas negativas, corta el aprendizaje.

9.11.5 Leaky ReLU

Leaky ReLU introduce una pendiente pequeña para $x < 0$:

$$g(x) = \begin{cases} x & \text{si } x \geq 0 \\ \alpha x & \text{si } x < 0 \end{cases}$$

donde α es un pequeño valor positivo (usualmente 0.01).

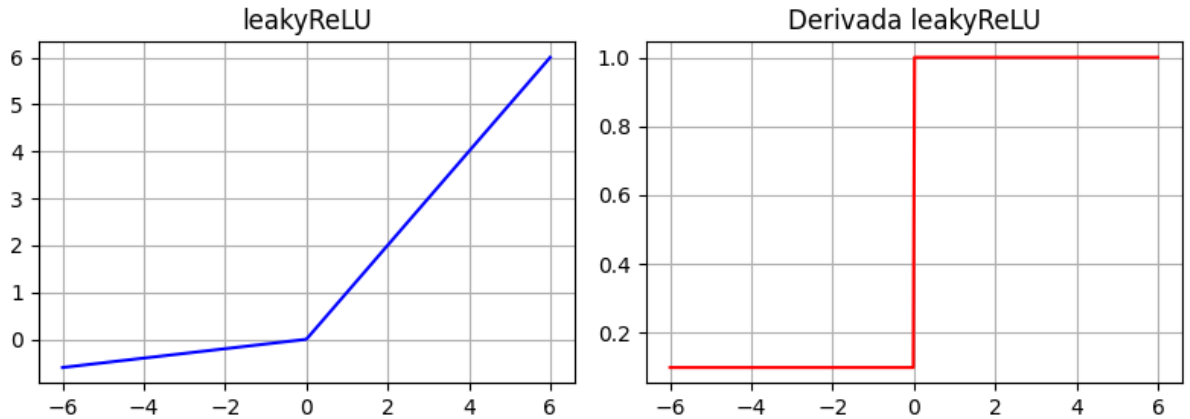


Figura 33: Función Leaky ReLU y su derivada

Al mantener una pendiente pequeña para valores negativos, Leaky ReLU ayuda a mitigar el problema de neuronas muertas de ReLU.

9.12 Regularización

Así como en 2.5, el objetivo de esta sección es la de reducción de varianza y consecuentemente, el sobreajuste. A diferencia de los modelos lineales, el sobreajuste en aprendizaje profundo es un problema mucho más serio, ya que se trata de modelos más complejos. A las técnicas ya mencionadas, se discuten algunas estrategias adicionales para disminuir la varianza del modelo [7].

9.12.1 Penalizaciones de norma en los parámetros

Es la forma más escuchada para regularizar y es la ya descrita en 2.5, con los métodos de Ridge, LASSO y ElasticNet.

9.12.2 Detención temprana (Early Stopping)

La detención temprana consiste en monitorear el rendimiento del modelo en un conjunto de validación durante el entrenamiento. Si el rendimiento en el conjunto de validación comienza a deteriorarse mientras que el rendimiento en el conjunto de entrenamiento sigue mejorando, se detiene el entrenamiento. Esto ayuda a prevenir el sobreajuste al evitar que el modelo se ajuste demasiado a los datos de entrenamiento. En la figura 34 se observa una detención temprana en torno a las 300 iteraciones.

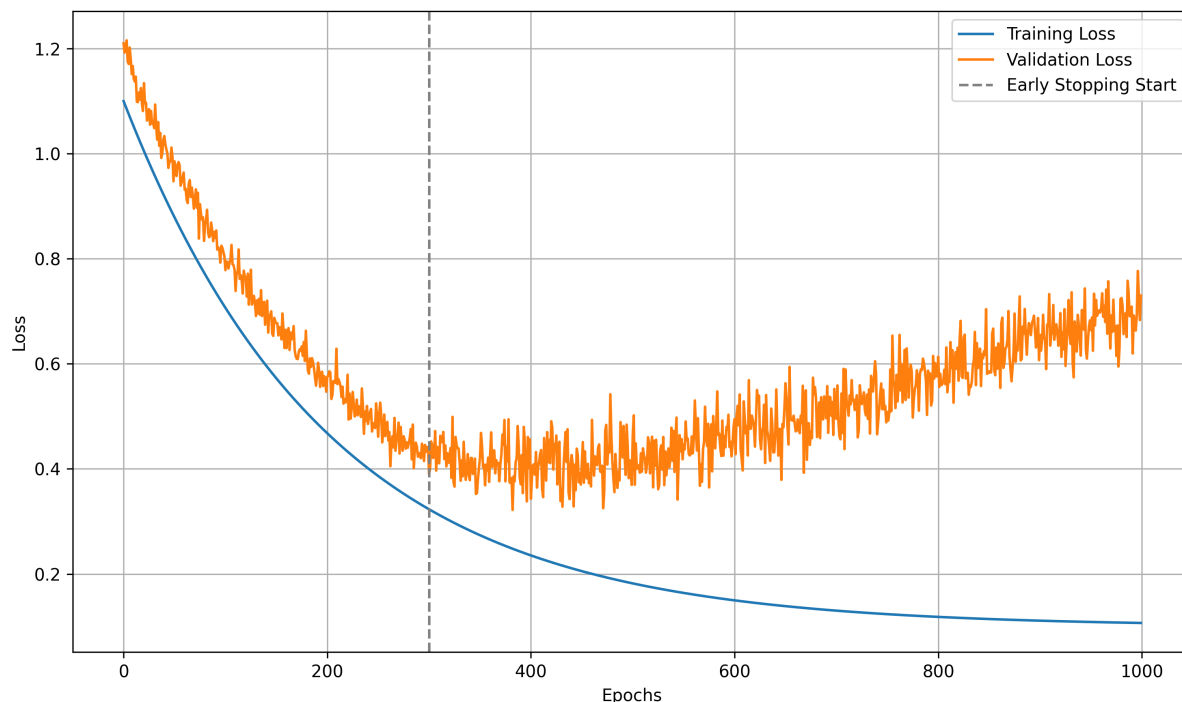


Figura 34: Ejemplo de curva de entrenamiento y validación con early stopping.

9.12.3 Aumento de datos (Data Augmentation)

El aumento de datos implica generar nuevas muestras de entrenamiento a partir de las existentes mediante transformaciones como rotaciones, desplazamientos, cambios de escala, entre otros. Esto aumenta la diversidad del conjunto de entrenamiento y ayuda a que el modelo generalice mejor a datos no vistos.

9.12.4 Regularización por ruido

Añadir ruido a las entradas o a las activaciones de las neuronas durante el entrenamiento puede ayudar a prevenir el sobreajuste. Esto obliga al modelo a aprender representaciones más robustas y menos sensibles a pequeñas variaciones en los datos de entrada. Se debe tener cuidado que el algoritmo no sobreajuste incluso más.

9.12.5 Ensamblado de modelos (Ensemble Learning)

De forma más general, y no asociado solamente a aprendizaje profundo, el ensamblado de modelos combina las predicciones de varios modelos para mejorar la precisión y la robustez. Algunas técnicas de ensamblado son:

- **Bagging**: entrenar múltiples modelos en diferentes subconjuntos de los datos de entrenamiento y promediar sus predicciones.
- **Boosting**: entrenar modelos secuenciales donde cada modelo intenta corregir los errores del anterior.
- **Stacking**: entrenar múltiples modelos y luego entrenar un modelo adicional para combinar sus predicciones.

9.12.6 Dropout

El dropout [18] es una técnica simple pero efectiva para realizar regularización. Consiste en desactivar aleatoriamente (con una probabilidad p) un subconjunto de neuronas durante cada paso de entrenamiento. Es decir, en cada iteración, cada neurona tiene una probabilidad $1 - p$ de mantenerse activa y p de ser “apagada” temporalmente, junto con sus conexiones entrantes y salientes, como se ilustra en la figura 35. Esto impide que la red dependa excesivamente de neuronas específicas y fuerza al modelo a aprender representaciones más robustas y redundantes.

Matemáticamente, dado un vector de activaciones $\mathbf{h}$ en una capa, se define un vector binario $\mathbf{r}$ donde cada elemento $r_i \sim \text{Bernoulli}(1 - p)$, y la salida con dropout es:

$$\tilde{\mathbf{h}} = \mathbf{r} \odot \mathbf{h}$$

donde $\odot$ denota el producto elemento a elemento.

Durante el entrenamiento, la red se ajusta con estas máscaras aleatorias que apagan neuronas. En cambio, durante la evaluación o inferencia, no se aplican “apagones”; en su lugar, se escala la activación de las neuronas por el factor p para mantener el valor esperado constante y evitar discrepancias entre entrenamiento e inferencia.

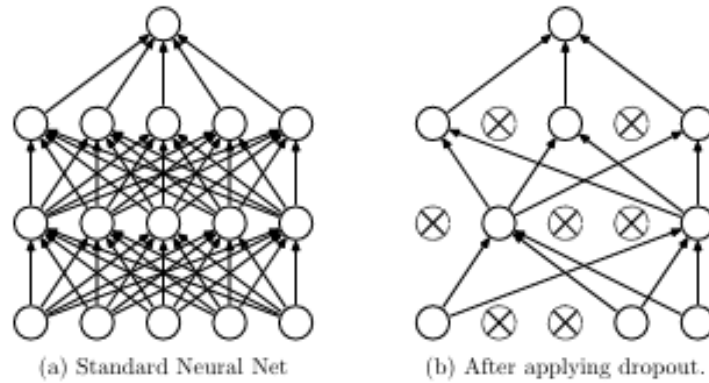


Figura 35: Esquema de dropout. A la izquierda, una red neuronal estándar con dos capas ocultas. A la derecha, un ejemplo de una red producida por aplicar dropout a la red de la izquierda. Algunas de las neuronas han sido seleccionadas para desconectarse.

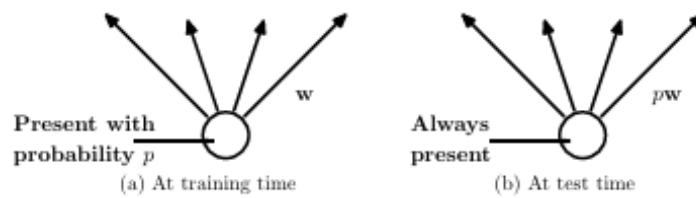


Figura 36: A la izquierda, el efecto de dropout en el entrenamiento: la neurona estará presente con una probabilidad p en cada iteración. A la derecha, el efecto en inferencia: la neurona estará presente, con los coeficientes promedio encontrados y escalados por p

Al evitar co-adaptaciones fuertes entre neuronas, el dropout actúa como una forma de ensamblaje implícito de múltiples redes neuronales.

En Srivastava et al.[18] demuestran que dropout mejora significativamente el rendimiento en múltiples benchmarks clásicos como tareas de reconocimiento de imágenes y texto, y supera a otras técnicas de regularización como la penalización $L2$. Adicionalmente, comentan que usualmente el dropout óptimo en las capas de entrada está cercano a 0, mientras que en capas ocultas puede ser hasta 0.5.

9.12.7 Ejemplos adversariales

Los ejemplos adversariales son entradas modificadas intencionalmente para "engañar" al algoritmo de aprendizaje y hacer que realice predicciones incorrectas. Entrenar modelos para ser robustos a estos ejemplos puede mejorar su capacidad de generalización [19].

9.13 Ajuste de hiperparámetros

El desempeño de una red neuronal depende en gran medida de la correcta elección de sus hiperparámetros. Sumado a la tasa de aprendizaje y el batch size, vistos en 2.4.4, otros ejemplos de hiperparámetros son el número de capas ocultas y el de neuronas por capa, la función de activación de cada capa oculta y la regularización empleada en cada capa (se recuerda que dropout tiene un parámetro p y $L2$ tiene un parámetro α).

Entre los métodos vistos en 8.2, los más utilizados suelen ser Random search y Optimización Bayesiana, dado que los tiempos de cómputo para una búsqueda exhaustiva como Grid search suelen ser demasiado elevados.

10 MLOps

El auge del aprendizaje automático en aplicaciones reales ha puesto de manifiesto la necesidad de prácticas robustas para el desarrollo, despliegue y mantenimiento de modelos. En este contexto surge **MLOps** (Machine Learning Operations), concepto introducido en 2015 [15].

Las operaciones de Machine Learning (MLOps) son las prácticas y herramientas que se utilizan en la gestión y el despliegue (deployment) en un entorno productivo, de forma eficiente, donde se combinan metodologías de ingeniería de software, desarrollo ágil y operaciones para asegurar la producción y operación efectiva de modelos de ML [13].

MLOps intenta garantizar que los modelos sean confiables, escalables y mantenibles en entornos productivos. Es una extensión de las prácticas tradicionales de DevOps, adaptada a las particularidades del desarrollo y operación de modelos de ML. Mientras que DevOps se enfoca en la entrega continua y mantenimiento de software, MLOps debe contemplar además la gestión de datos, la reproducibilidad de experimentos, el versionado de modelos y la monitorización del desempeño de los modelos en producción [2] [1].

El objetivo principal de MLOps es cerrar la brecha entre el equipo de Data Science y el de ingeniería/operaciones, a través de automatizar procesos y establecer buenas prácticas que permitan iterar rápidamente sin perder control sobre la calidad y estabilidad del sistema.

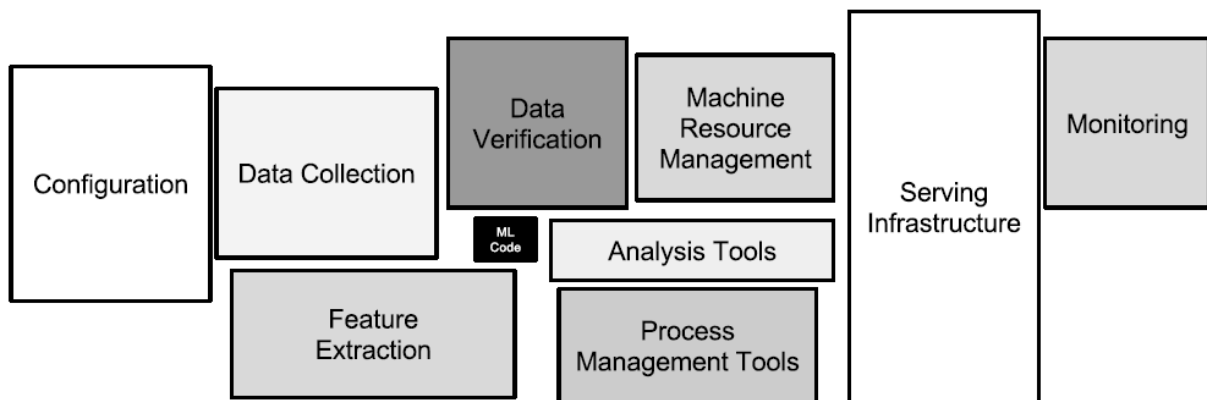


Figura 37: Sólo una pequeña fracción de un sistema real de ML está compuesto por el código del modelo propiamente dicho. La infraestructura requerida para la puesta en producción puede ser muy compleja.

En las siguientes secciones se profundizará en los aspectos fundamentales de MLOps, sus niveles de madurez, herramientas, y desafíos asociados a su implementación.

10.1 Entornos de Desarrollo y Producción

El entorno de desarrollo y el entorno de producción presentan características y requisitos muy distintos.

Entorno de desarrollo Es el espacio donde los científicos de datos experimentan, diseñan y entrenan modelos. En este entorno, la flexibilidad, la experimentación rápida y la iteración continua son prioritarias. Se utilizan herramientas interactivas, librerías para análisis exploratorio y frameworks de modelado. La reproducibilidad puede ser más flexible, y los recursos computacionales suelen estar orientados a acelerar el desarrollo.

Entorno de producción Es donde los modelos se despliegan para ser consumidos por aplicaciones, usuarios o sistemas automatizados. En producción, la estabilidad, escalabilidad, seguridad y eficiencia son fundamentales. Los modelos deben responder con baja latencia, manejar cargas variables y ser monitoreados para detectar fallas o degradación en su desempeño. Las tareas alojadas en este entorno ya fueron validadas para un modelo de negocio. Cuentan con seguridad más rigurosa que un entorno de desarrollo.

Sólo los modelos productivos aportan real valor. El foco del desarrollo es llegar a la instancia de producción, por lo que se debe simular, lo más fiel posible, las condiciones que afrontará el algoritmo una vez desplegado. Luego, el monitoreo del desempeño del modelo es clave para evitar sufrir el fenómeno de *deriva de datos*, que se describe en 10.2.1.

10.1.1 Versionado de modelos

El versionado de modelos tiene como objetivo mantener la trazabilidad y reproducibilidad de los resultados. Permite comparar distintas versiones del modelo, revertir cambios si es necesario, y auditar el proceso de desarrollo. Algunos aspectos clave incluyen:

- Guardar el estado completo del modelo, incluyendo arquitectura, pesos, hiperparámetros y métricas.
- Asociar metadatos relevantes como fecha, autor, datos usados para entrenamiento, y entorno de ejecución.
- Utilizar sistemas de control de versiones específicos para modelos o integrarlos con sistemas de control de código fuente.

Existen herramientas como MLflow, DVC y TensorBoard que facilitan esta tarea.

10.1.2 Despliegue y automatización

El despliegue consiste en poner el modelo en producción para que pueda ser consumido por aplicaciones o usuarios finales. La automatización de este proceso garantiza rapidez, consistencia y reducción de errores humanos. Algunas prácticas recomendadas pueden ser:

- Uso de contenedores (Docker) para empaquetar el modelo y su entorno.
- Orquestación mediante Kubernetes u otras plataformas para escalabilidad y disponibilidad.
- Integración con pipelines CI/CD que automaticen pruebas, validaciones y despliegues (secciones 10.3.1 y 10.3.2).

- Implementación de APIs REST.
- Monitoreo continuo del rendimiento y uso para detectar fallas o degradación (sección 10.2.1).

10.2 Monitoreo y mantenimiento

Una vez que un modelo está en producción, se debe monitorear su desempeño para asegurar que sigue siendo útil y confiable. No solo para evitar el data drift, sino también errores en la infraestructura (disponibilidad, latencia, entre otros).

Para el mantenimiento, se establecen mecanismos de alerta, análisis automático y ciclos de retraining (reentrenamiento) para actualizar el modelo cuando sea necesario.

10.2.1 Deriva de datos (data drift)

La *deriva de datos* o *data drift* se refiere a los cambios en la distribución estadística de los datos de entrada a un modelo a lo largo del tiempo. Estos cambios pueden causar que el modelo pierda precisión y confiabilidad, ya que fue entrenado con datos que ya no representan adecuadamente el entorno actual.

Las causas comunes incluyen:

- cambios en el comportamiento del usuario o cliente.
- variaciones estacionales o temporales. Eventos.
- cambios en los sensores o sistemas de captura de datos.
- actualizaciones en el proceso de negocio o producto.
- errores en el diseño del dataset.

La detección de la deriva es el primer paso para tomar una decisión sobre su mitigación. Algunas posibilidades son:

- comparación de distribuciones estadísticas.
- métricas de distancia entre distribuciones.
- monitoreo de estadísticas resumidas (media, varianza) en ventanas temporales.
- uso de modelos no supervisados para detectar cambios en las características.

Una vez detectada la deriva, se pueden tomar acciones de mitigación como:

- reentrenamiento periódico o basado en eventos.
- ajuste dinámico del modelo mediante aprendizaje continuo y evitar seleccionar features manualmente.
- implementación de modelos adaptativos o ensembles.

La deriva usualmente se puede ver mitigada por el monitoreo automatizado y la capacidad de respuesta rápida.

10.2.2 Alertas y retrain

El monitoreo continuo del desempeño del modelo en producción debe estar acompañado de mecanismos automáticos de alerta que notifiquen al equipo cuando se detectan anomalías, como deriva de datos o caída en la métrica clave elegida [20].

Estas alertas permiten activar procesos de *retrain* (reentrenamiento), donde el modelo se actualiza usando datos nuevos para recuperar o mejorar su rendimiento. Este reentrenamiento puede ser periódico, donde se programa para realizarse a intervalos regulares, por ejemplo, semanal o mensual. Puede estar basado en eventos, donde el reentrenamiento se activa automáticamente cuando las alertas indican una degradación significativa o directamente puede ser manual, donde el equipo decide cuándo lanzar el proceso tras analizar las alertas.

Además, es recomendable implementar un ciclo de validación para evaluar la nueva versión del modelo antes de su despliegue, asegurando problemas inesperados.

10.3 Niveles de MLOps

En esta sección, se toma un interesante concepto definido por Google [8] para clasificar en tres niveles a las prácticas de MLOps. Antes de definir los niveles, es necesario entender qué es integración continua y entrega continua en desarrollo de software.

10.3.1 Integración continua

La integración continua (Continuous Integration, CI) es la práctica de integrar el trabajo de varios desarrolladores de forma frecuente (varias veces al día) en un repositorio compartido. En MLOps, CI implica:

- versionado del código y scripts de entrenamiento.
- pruebas automáticas para validar el código y la calidad de datos.
- validación de dependencias y entornos de ejecución reproducibles.
- posible ejecución automática de experimentos unitarios para verificar que cambios en el código no rompen el flujo de entrenamiento.

10.3.2 Entrega continua

La entrega continua (Continuous Delivery, CD) es la capacidad de desplegar cambios en el sistema de forma automática y segura, con la mínima intervención manual. En MLOps esto se asocia a:

- automatizar el empaquetado del modelo (por ejemplo, como servicio REST o microservicio).
- implementar pipelines de despliegue que incluyan pruebas de integración.
- garantizar que los nuevos modelos pasan validaciones antes de llegar a producción.
- posibilidad de revertir un despliegue en caso de degradación de rendimiento.

10.3.3 Nivel 0

El **Nivel 0** de MLOps se caracteriza por un flujo manual y aislado.

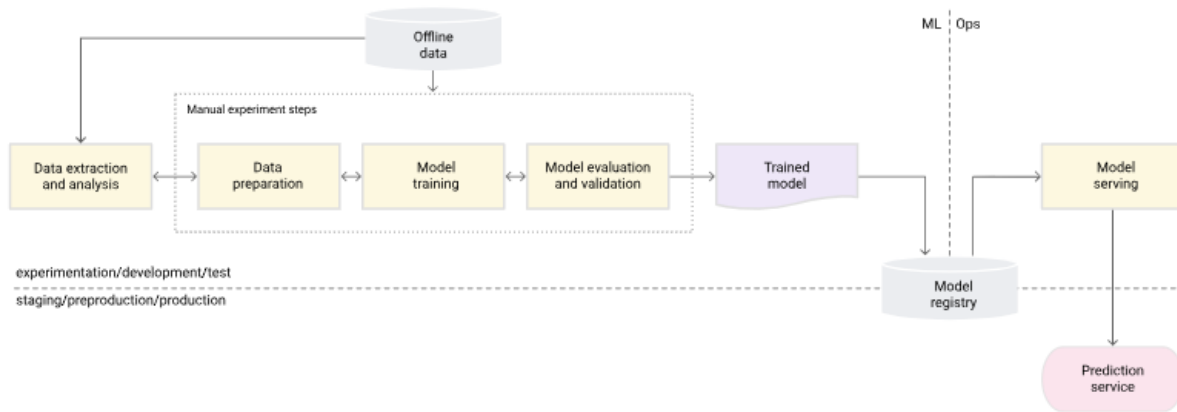


Figura 38: Nivel 0 de MLOps.

- El entrenamiento, validación y despliegue son procesos manuales.
- No hay pipelines automatizados.
- Los modelos se entrenan de forma esporádica y el monitoreo es mínimo.
- Alto riesgo de *drift* en datos y pérdida de rendimiento sin detección temprana.

Es común en entornos experimentales o prototipos.

10.3.4 Nivel 1

El **Nivel 1** introduce la **automatización del entrenamiento** y pruebas.

- Se definen pipelines reproducibles para entrenamiento y validación.
- Se implementa CI/CD básico para el código de ML.
- Hay seguimiento de experimentos y versionado de modelos.
- El despliegue aún puede requerir intervención manual.

Permite reducir errores humanos y acelerar iteraciones de desarrollo.

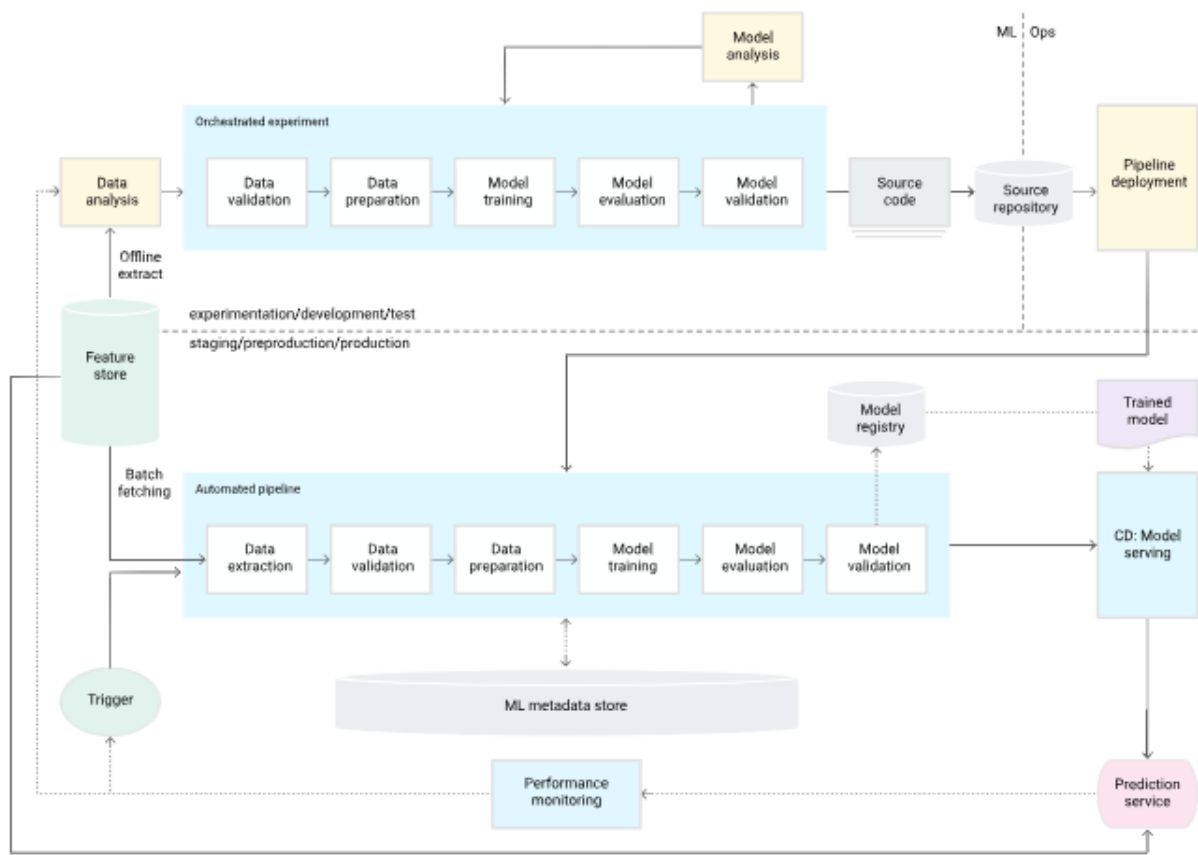


Figura 39: Nivel 1 de MLOps.

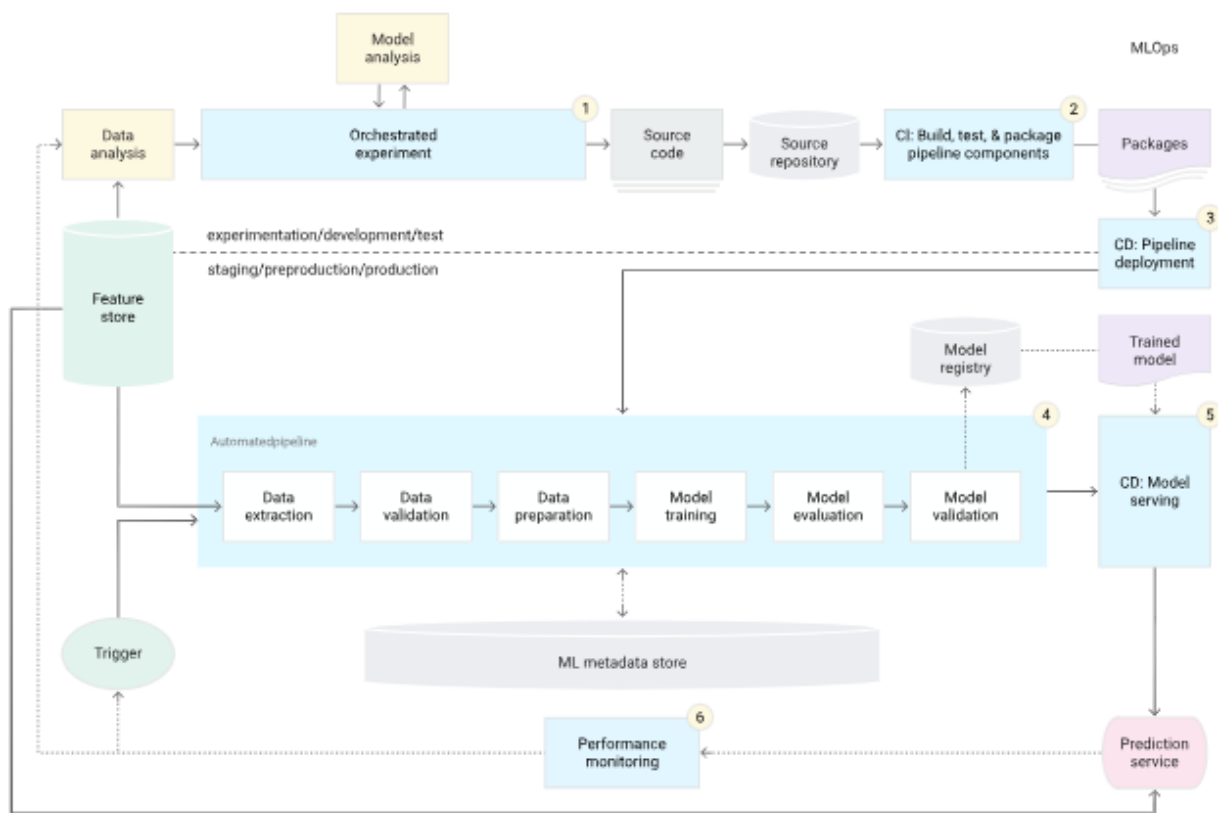


Figura 40: Nivel 2 de MLOps.

10.3.5 Nivel 2

El **Nivel 2** apunta a la automatización completa del ciclo de vida.

- CI/CD aplicados tanto al código como a los modelos y datos.
- *Pipelines* que van desde la ingesta de datos hasta el despliegue y monitoreo en producción.
- Monitoreo continuo de rendimiento y reentrenamiento automático.
- Alertas y triggers que detectan *data drift*.
- Integración de validaciones en tiempo real antes de actualizar el modelo en producción.

Este nivel permite escalabilidad, confiabilidad y rápida adaptación a cambios en los datos.

References

- [1] Saleema Amershi, Andrew Begel, Christian Bird, Robert DeLine, Harald Gall, Ece Kamar, Nikhil Nagappan, Besmira Nushi, Carolyn Selby, and Thomas Zimmermann. Software engineering for machine learning: A case study. In *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, pages 291–300. IEEE, 2019.
- [2] Daniel Baylor, Eric Breck, Henry Cheng, Noah Fiedel, Chansoo Haque, Minh Hoang, Lukasz Koc, Quoc V Pham, Vijay Vasudevan, Mike Young, et al. Tfx: A tensorflow-based production-scale machine learning platform. *arXiv preprint arXiv:1712.05855*, 2017.
- [3] James Bergstra and Yoshua Bengio. Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13(Feb):281–305, 2012.
- [4] Andriy Burkov. *The Hundred-Page Machine Learning Book*. Andriy Burkov, 2019.
- [5] Nitesh V Chawla, Kevin W Bowyer, Lawrence O Hall, and W Philip Kegelmeyer. Smote: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16:321–357, 2002.
- [6] Real Academia Española. Diccionario de la lengua española, 2024.
- [7] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. <http://www.deeplearningbook.org>.
- [8] Google Cloud. Mlops: Continuous delivery and automation pipelines in machine learning. <https://cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning>, 2021.

- [9] imbalance learn. imbalance-learn documentation, 2024. Último acceso: 5 de agosto de 2025.
- [10] Gareth James, Daniela Witten, Trevor Hastie, and Robert Tibshirani. *An Introduction to Statistical Learning: with Applications in R*. Springer, New York, NY, 1st edition, 2013.
- [11] Stephen Marsland. *Machine Learning: An Algorithmic Perspective*. CRC Press, Chapman ‘I&’ Hall, Boca Raton, FL, 2nd edition, 2015.
- [12] Christoph Molnar. *Interpretable Machine Learning*. Christoph Molnar, 3 edition, 2025.
- [13] Tim Mucci and Cole Stryker. What is mlops? <https://www.ibm.com/think/topics/mlops>, 2024. IBM Think Blog.
- [14] Scikit-learn. scikit-learn documentation, 2024. Último acceso: 18 de marzo de 2025.
- [15] D Sculley, G Holt, D Golovin, E Davydov, T Phillips, D Ebner, V Chaudhary, M Young, JF Crespo, and D Dennison. Hidden technical debt in machine learning systems. *Advances in Neural Information Processing Systems*, 28, 2015.
- [16] Bobak Shahriari, Kevin Swersky, Ziyu Wang, Ryan P Adams, and Nando De Freitas. Taking the human out of the loop: A review of bayesian optimization. *Proceedings of the IEEE*, 104(1):148–175, 2016.
- [17] SHAP developers. SHAP documentation. <https://shap.readthedocs.io/en/latest/index.html>, 2018. Último acceso: 5 de agosto de 2025.
- [18] Nitish Srivastava, Geoffrey E. Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(1):1929–1958, 2014.
- [19] Christian Szegedy, Wojciech Zaremba, Ilya Sutskever, Juan Bruna, Dumitru Erhan, Ian Goodfellow, and Rob Fergus. Intriguing properties of neural networks. In *International Conference on Learning Representations (ICLR) 2014*, 2014.
- [20] Swamy Prasad Rao Velaga. Real-time monitoring and retraining of ai models using continuous integration techniques. *International Journal of Research and Analytical Reviews (IJRAR)*, 11(2):559–567, 2024.